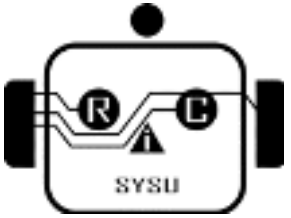


中山大學
SUN YAT-SEN UNIVERSITY



计算机组成原理

第四章：MIPS CPU逻辑设计

中山大学计算机学院
陈刚

2024年秋季

本节概要

重点内容

第四章 处理器

- 4.9 异常处理
- 4.10 指令级并行

基本要求

- 掌握异常的基本概念
- 理解处理器中的异常处理

4.9 异常

4.9 异常和中断的概念



- 程序执行的过程中，CPU会遇到一些特殊情况使正在执行的程序被“中断”
  - CPU中止原来正在执行的程序，转去处理异常情况或特殊事件的程序去执行，执行后再返回到原被中止的程序继续执行
  - 控制器设计中最难的部分：异常和中断的实现

异常和中断的处理



➢使程序执行被“中断”的事件有两类

■内部“异常”：CPU内发生的意外事件或特殊事件

- 按发生原因分为硬故障中断和程序性中断两类

①硬故障中断：如电源掉电、硬件线路故障等

②程序性中断：执行某条指令时发生的“异常(Exception)”，如溢出/缺页/越界/越权/非法指令/除数为0/断点/单步/系统调用等

- 按处理方式分故障(fault)、自陷(Trap)和终止(Abort)三类

①故障：执行指令引起的异常事件，如溢出/缺页/访问超时

②自陷：预先安排的事件，如单步跟踪、系统调用等(自愿中断)

③终止：硬故障事件，机器将终止，调出中断程序重启操作系统

自陷处理完后回到哪条指令执行？ 回到下条指令！

异常和中断的处理



➢使程序执行被“中断”的事件有两类

■内部“异常”：CPU内发生的意外事件或特殊事件

- 按发生原因分为硬故障中断和程序性中断两类

①硬故障中断：如硬件线路故障等

②程序性中断：执行某条指令时发生的“异常(Exception)”，如溢出/缺页/越界/越权/非法指令/除数为0/断点/单步/系统调用等

- 按处理方式分故障(fault)、自陷(Trap)和终止(Abort)三类

①故障：执行指令引起的异常事件，如溢出/缺页/访问超时

思考：哪些故障补救后可继续执行，哪些只好终止当前进程？

① 缺页等：补救后可继续，回到发生故障的指令重新执行

② 溢出、除数为0、非法操作、内存保护错等：终止当前进程

③ 自陷：回到当前指令的下一条指令执行

异常和中断的处理



- 使程序执行被“中断”的事件有两类
  - 内部“异常”：CPU内发生的意外事件或特殊事件
    - 按发生原因分为硬故障中断和程序性中断两类

    ①硬故障中断：如电源掉电、硬件线路故障等

    ②程序性中断：执行某条指令时发生的“异常(Exception)”，如溢出/缺页/越界/越权/非法指令/除数为0/断点/单步/系统调用等
    - 按处理方式分故障(fault)、自陷(Trap)和终止(Abort)三类
  - 外部“中断”：CPU外发生的特殊事件，由“中断请求”信号发给CPU请求处理
    - 如实时钟、打印机缺纸、外设准备好、采样计时到、DMA传输结束等

实例—8086/8088中断系统

统称为“中断”：内中断(内部异常)和外中断(外部中断)

- 内中断：CPU自己产生而不通过中断请求线请求，皆为不可屏蔽中断
  - 指令引起异常：CPU执行预置指令后在特定情况下发生的异常
    - INTO 溢出：执行算术指令后，若发生溢出，则产生类型4中断
    - INT n 用户定义：指令的第二字节给出一个类型号(n=0～255)。例如：n=3 (INT 3)时为断点设置，该指令执行后自动产生类型3中断

实例—8086/8088中断系统



统称为“中断”：内中断(内部异常)和外中断(外部中断)

- 内中断：CPU自己产生而不通过中断请求线请求，皆为不可屏蔽中断
  - 指令引起异常：CPU执行预置指令后在特定情况下发生的异常
    - INTO 溢出
    - INT n 用户定义
  - 处理器检测异常：CPU执行指令时产生的异常
    - 如：除法错、无效操作码、缺页、单步跟踪调试等
    - 除法错：除数为0或商溢出，则产生类型0中断
    - 单步跟踪：当自陷位TF=1时，每条指令执行完就自动产生类型1中断

实例—8086/8088中断系统

- 外中断：通过中断请求线INTR和NMI来实现
  - INTR(Interrupt Require)
    - 可屏蔽中断(外设中断源引起的中断)
    - CPU可以响应、也可以不响应：受本身的屏蔽位、以及CPU标志寄存器中的中断允许位(IF)控制

      ①置IF=1：指令STI——开中断

      ②清IF=0：指令CLI——关中断
  - NMI(Nonmaskable Interrupt)：不可屏蔽中断(重要或紧急的硬件故障)，属于类型2中断
    - CPU必须无条件响应
    - 如电源掉电、存储器读写出错、总线奇偶位出错等

实例—8086/8088中断系统



- **外中断：通过中断请求线INTR和NMI来实现**
  - **INTR(Interrupt Require)**
    - 可屏蔽中断(外设中断源引起的中断)
    - CPU可以响应、也可以不响应：受本身的屏蔽位、以及CPU标志寄存器中的中断允许位(IF)控制
      - ① 置IF=1：指令STI——开中断
      - ② 清IF=0：指令CLI——关中断
  - **NMI(Nonmaskable Interrupt)**：不可屏蔽中断(重要或紧急的硬件故障)，属于类型2中断
    - CPU必须无条件响应

所有事件都被分配一个“中断类型号”

每个中断都有相应的“中断服务程序”

可根据中断类型号找到中断服务程序的入口地址

处理器中的异常处理机制



- 检测到异常时，处理器必须进行以下基本处理

① 关中断：使处理器处于“禁止中断”状态，以防止新异常(或中断)破坏断点和现场

② 保护断点和程序状态：将断点和程序状态保存到堆栈或特殊寄存器中

③ 识别异常事件：软件识别和硬件识别

处理器中的异常处理机制



- 检测到异常时，处理器进行基本处理
  - ①关中断：使处理器处于“禁止中断”状态，以防止新异常(或中断)破坏断点和现场
  - ②保护断点和程序状态：将断点和程序状态保存到堆栈或特殊寄存器中
    - PC→堆栈 或 EPC(专门存放断点的寄存器)
    - PSWR →堆栈 或 EPSWR (专门保存程序状态的寄存器)
      - PSW—Program Status Word：程序状态字，包括条件码、中断码、状态位等
      - PSWR(PSW寄存器)：用于存放程序状态字寄存器。如X86的FLAGS

处理器中的异常处理机制



- 检测到异常时，处理器进行基本处理
  - ①关中断
  - ②保护断点和程序状态
  - ③识别异常事件：软件识别和硬件识别
    - 软件识别(非向量中断、MIPS采用)

      设置一个异常状态寄存器(如MIPS中为Cause寄存器)，用于记录异常原因。操作系统用一个异常处理程序，按优先级顺序查询异常状态寄存器，识别出异常事件，转入相应的中断服务程序执行
    - 硬件识别(向量中断、80x86采用)

      用专门的硬件查询电路按优先级顺序识别异常，得到“中断类型号”，到中断向量表中读取对应的中断服务程序的入口地址

8086/8088的中断向量表

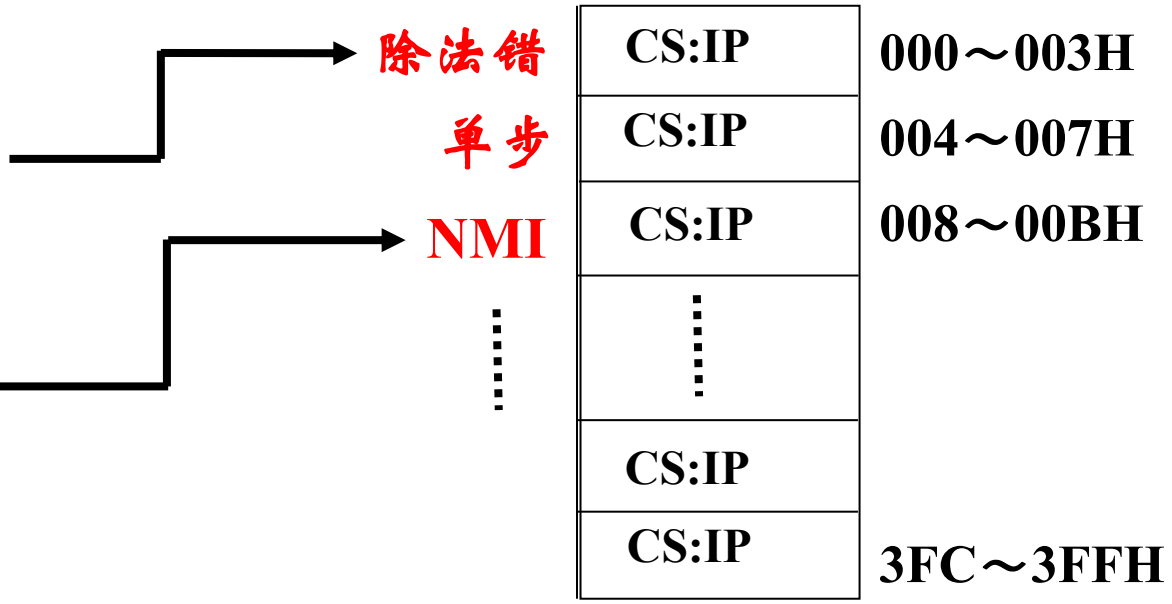
- 中断向量表（中断入口地址表），位于0000H～03FFH
- 共含256个中断向量,每一项就是对应的中断服务程序的入口地址，每个向量长度为四个字节(CS:IP)
- 中断向量地址=中断类型号×4

8086/8088的中断向量表

- 中断向量表（中断入口地址表），位于0000H～03FFH。
- 共含256个中断向量,每一项就是对应的中断服务程序的入口地址，每个向量长度为四个字节(CS:IP)。
- 中断向量地址=中断类型号×4

例1：除法错的中断类型号为0，
向量地址为：0×4=0

例2：NMI的中断类型号为2，
向量地址为：2×4=8



若已知一个中断类型码，需要通过两次地址转换：
- 中断类型码到中断向量表地址
- 中断向量表地址到中断处理程序地址

实例——MIPS带异常处理的数据通路设计



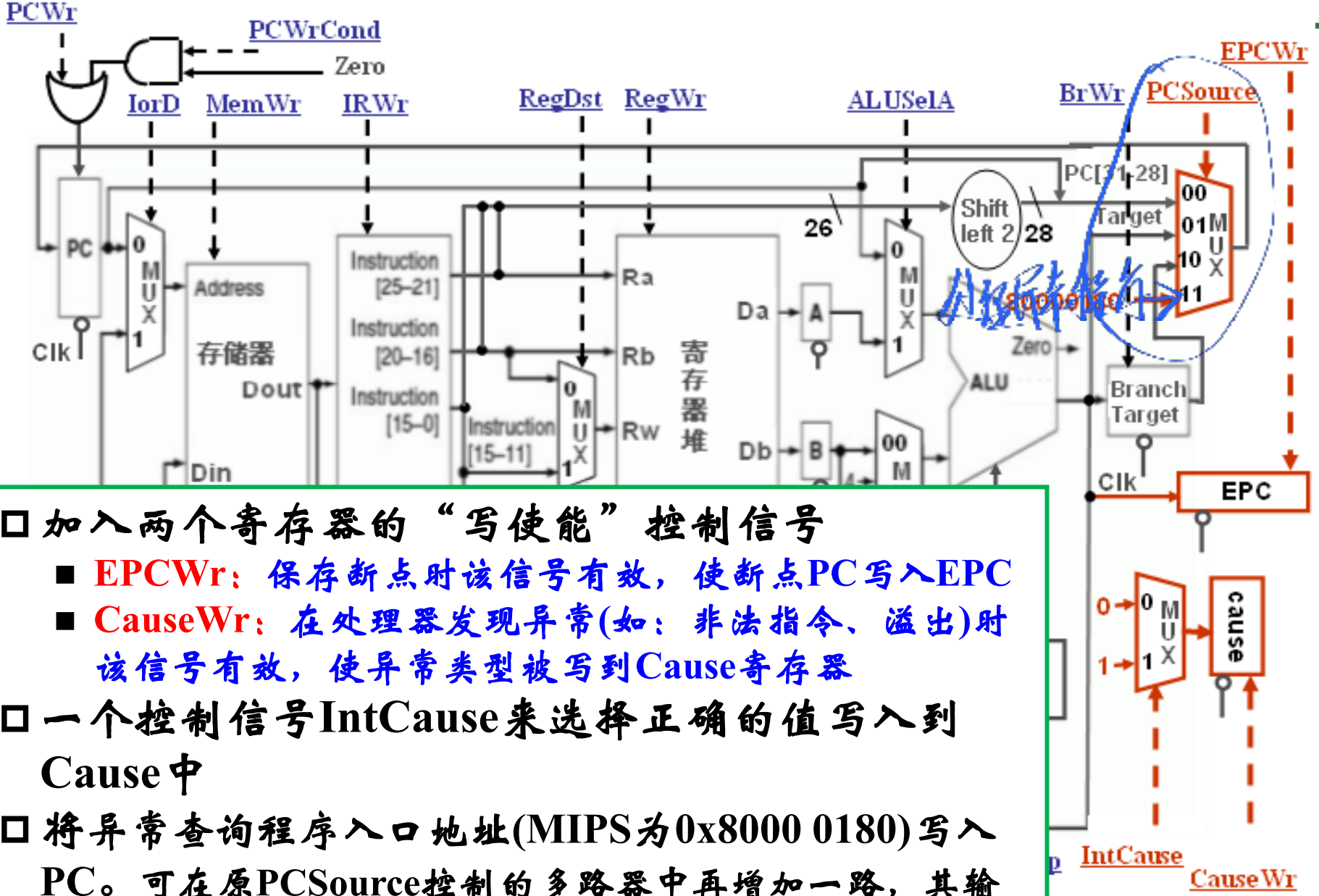
- MIPS采用软件识别中断源
  - 操作系统提供的一个特定的异常查询程序
- 数据通路中需增加两个寄存器
  - EPC：32位，用于存放断点(异常处理后返回到的指令的地址)
    - 写入EPC的断点可能是正在执行的指令(故障时)：把PC的值减4后送到EPC
    - 写入EPC的断点可能是下条指令(自陷和中断时)：直接送PC到EPC
  - Cause：32位，记录异常原因
    - 处理的异常类型：例如未定义指令(Cause=10)、算术溢出(Cause=12)

MIPS带异常处理的数据通路设计



- MIPS采用软件识别中断源
  - 操作系统提供的一个特定的异常查询程序
- 数据通路中需增加两个寄存器
  - EPC：32位，用于存放断点(异常处理后返回到的指令的地址)
  - Cause：32位，记录异常原因
- 需要加入两个寄存器的“写使能”控制信号
  - EPCWr：在保存断点时该信号有效，使断点PC写入EPC
  - CauseWr：在处理器发现异常(如非法指令等)时，该信号有效，使异常类型被写到Cause寄存器
- 需要一个控制信号IntCause：选择正确的值写入到Cause中
- 需要将异常查询程序的入口地址(MIPS为0x8000 0180)写入PC：在原来PCSource控制的多路复用器中再增加一路，其输入为0x8000 0180

带异常处理的多周期数据通路



- 加入两个寄存器的“写使能”控制信号
  - EPCWr：保存断点时该信号有效，使断点PC写入EPC
  - CauseWr：在处理器发现异常(如：非法指令、溢出)时该信号有效，使异常类型被写到Cause寄存器
- 一个控制信号IntCause来选择正确的值写入到Cause中
- 将异常查询程序入口地址(MIPS为0x8000 0180)写入PC。可在原PCSource控制的多路器中再增加一路，其输入为：0x8000 0180

流水线方式下的异常和中断处理



- 异常和中断都会改变程序的执行顺序
- 某条指令发现异常时，后面多条指令已被取到流水线中执行

  如ALU指令发现“溢出”时，已经到EX阶段结束了，此时，其后面已有两条指令进入流水线

流水线方式下的异常和中断处理



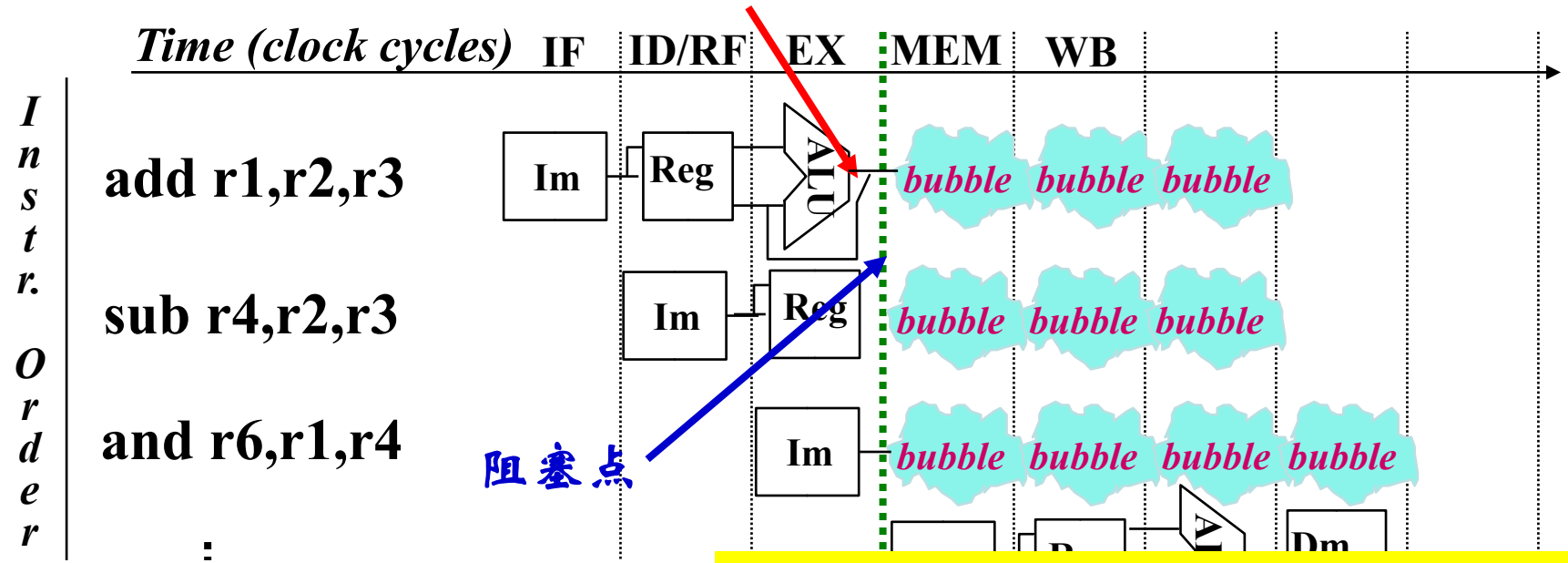
➢ 流水线数据通路如何处理异常？

- 举例：假设指令add r1,r2,r3产生了溢出
- 处理思路：
  - 清除add指令以及后面的所有已在流水线中的指令
  - 关中断(将CPU的中断允许触发器清0)
  - 保存断点（PC或PC+4）到 EPC
  - 0x8000 0180送PC(从0x8000 0180处开始取指令)

注意：MIPS异常处理程序的首地址为0x8000 0180

异常的处理

➢异常(溢出)在第一条指令的EXE阶段被检出



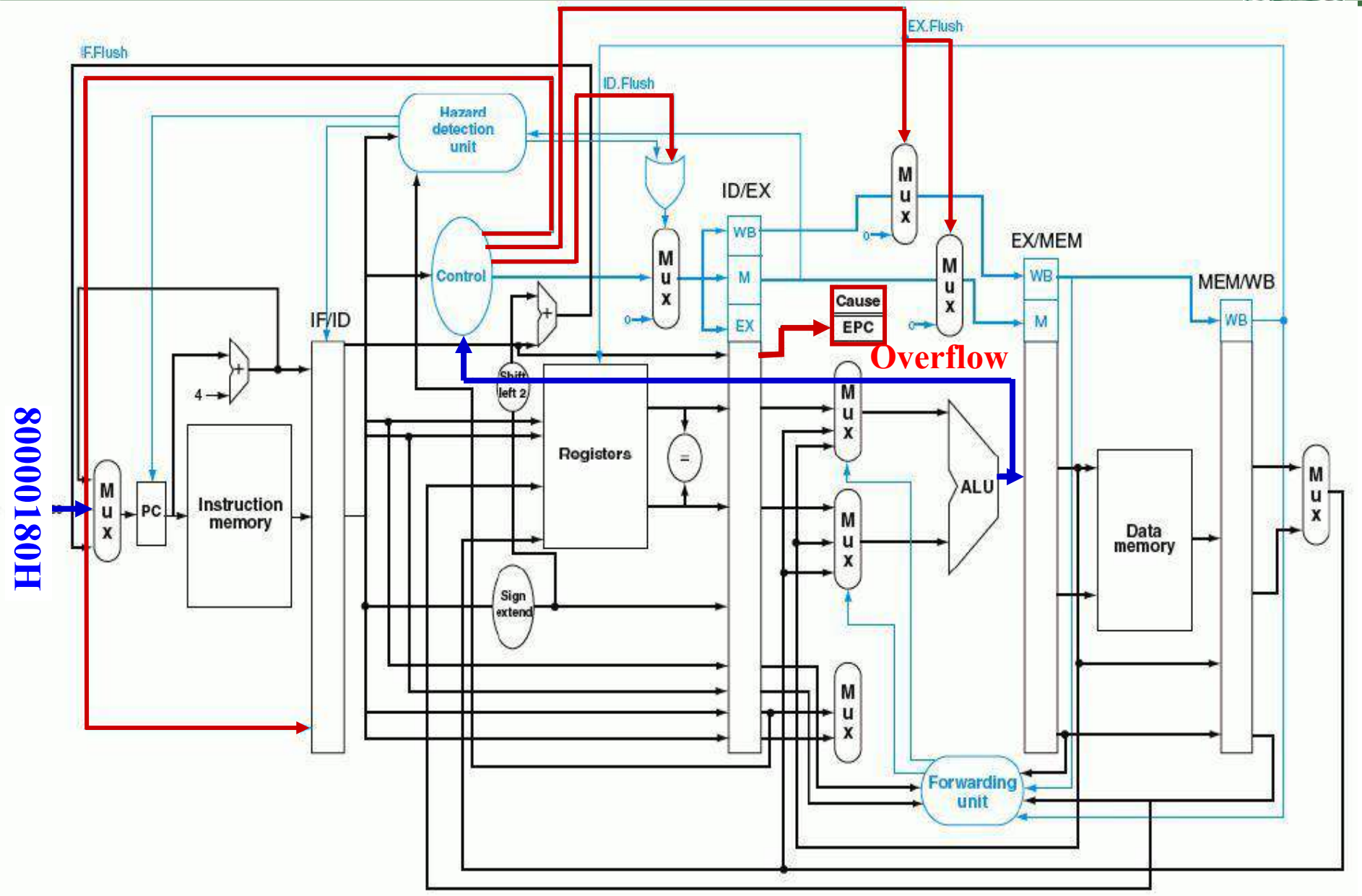
问题：会发生将溢出结果写到寄存器r1中去的情况吗？

不会！EX.Flush使EXE段指令的控制信号清0(主要保证RegWr清0)，避免了在WB阶段写寄存器r1的情况

■ 异常处理：

- IF.Flush使IF段指令在IF/ID寄存器中清为0，变成nop指令
- ID.Flush与数据冒险阻塞检测信号or后，使ID段指令的控制信号清0
- EX.Flush使EX段指令的控制信号清0
- 关中断，并将断点(PC或者PC+4)保存到EPC中
- 将0x8000 0180作为PC的一个输入，并控制PC输入端的多路选择器

带异常处理的流水线数据通路



流水线方式下的异常处理的难点问题



➢1、流水线中同时有5条指令，哪一条发生异常？

■ 根据异常发生的流水段可确定是哪条指令，因为各类异常发生的流水段不同 →从而可知哪条指令异常

- “溢出”在EX段检出
- “无效指令”在ID段检出
- “除数为0”在ID段检出
- “无效指令地址”在IF段检出
- “无效数据地址”在Load/Store指令的EX段检出

➢2、外部中断与特定指令无关，如何确定处理点？

■ 可在IF段或WB段中进行中断查询，需要保证当前WB段的指令能正确完成，并在有中断发生时，确保下个时钟开始执行中断服务程序

流水线方式下的异常处理的难点问题



- 3、检测到异常时，指令已经取出多条，当前PC的值已不是断点，怎么办？
  - 指令地址存放在流水段寄存器中，把这个地址送到EPC保存，以实现精确中断
    - 精确中断(precise interrupt)或精确异常(precise exception)
      - 对于发生中断或者异常的单条指令，中断被称为“精确”当且仅当满足：所有该指令之前的指令都已经提交其状态；所有后继指令(包括中断的指令)没有改变任何机器的状态
      - 在流水线计算机中，异常或中断与其产生的指令始终能精确地关联起来

流水线方式下的异常处理的难点问题

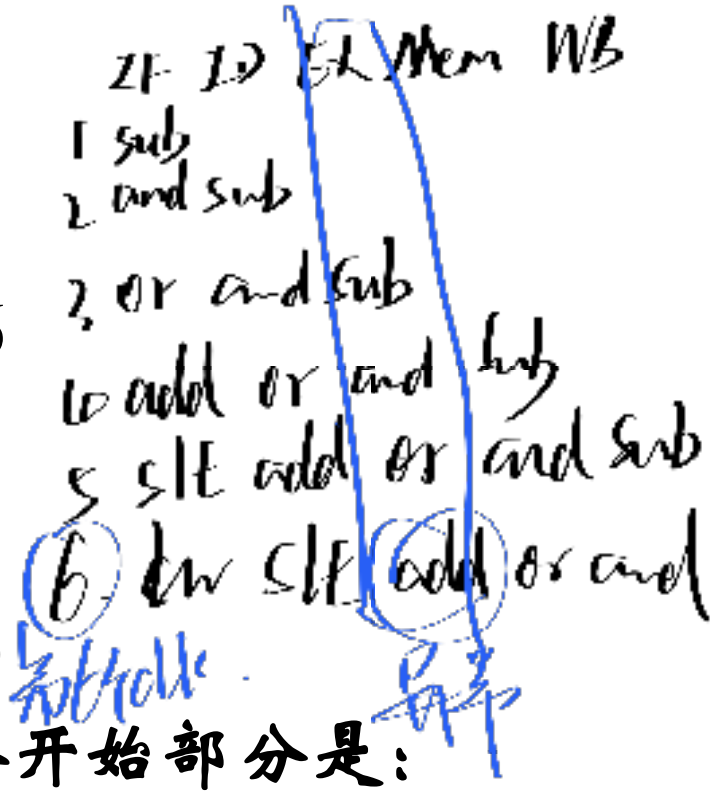


- 4、一个时钟周期内可能有多个异常，先处理哪个？
  - 在中断查询程序或中断优先级排队电路中按序查询
- 5、系统中只有一个EPC，多个异常发生时，一个EPC不够放多个断点，怎么办？
  - 总是把优先级最高的保存到EPC中
- 6、异常处理过程中，又发生新异常或中断，怎么办？
  - 利用中断屏蔽和中断嵌套机制来处理

举例——流水线计算机中的异常

例：给定如下指令序列

40_{hex}	sub	$11,	$2,	$4
44_{hex}	and	$12,	$2,	$5
48_{hex}	or	$13,	$2,	$6
$4C_{hex}$	add	$1,	$2,	$1
50_{hex}	slt	$15,	$6,	$7
54_{hex}	lw	$16,	50($7)	



假设在异常发生时，执行的指令开始部分是：

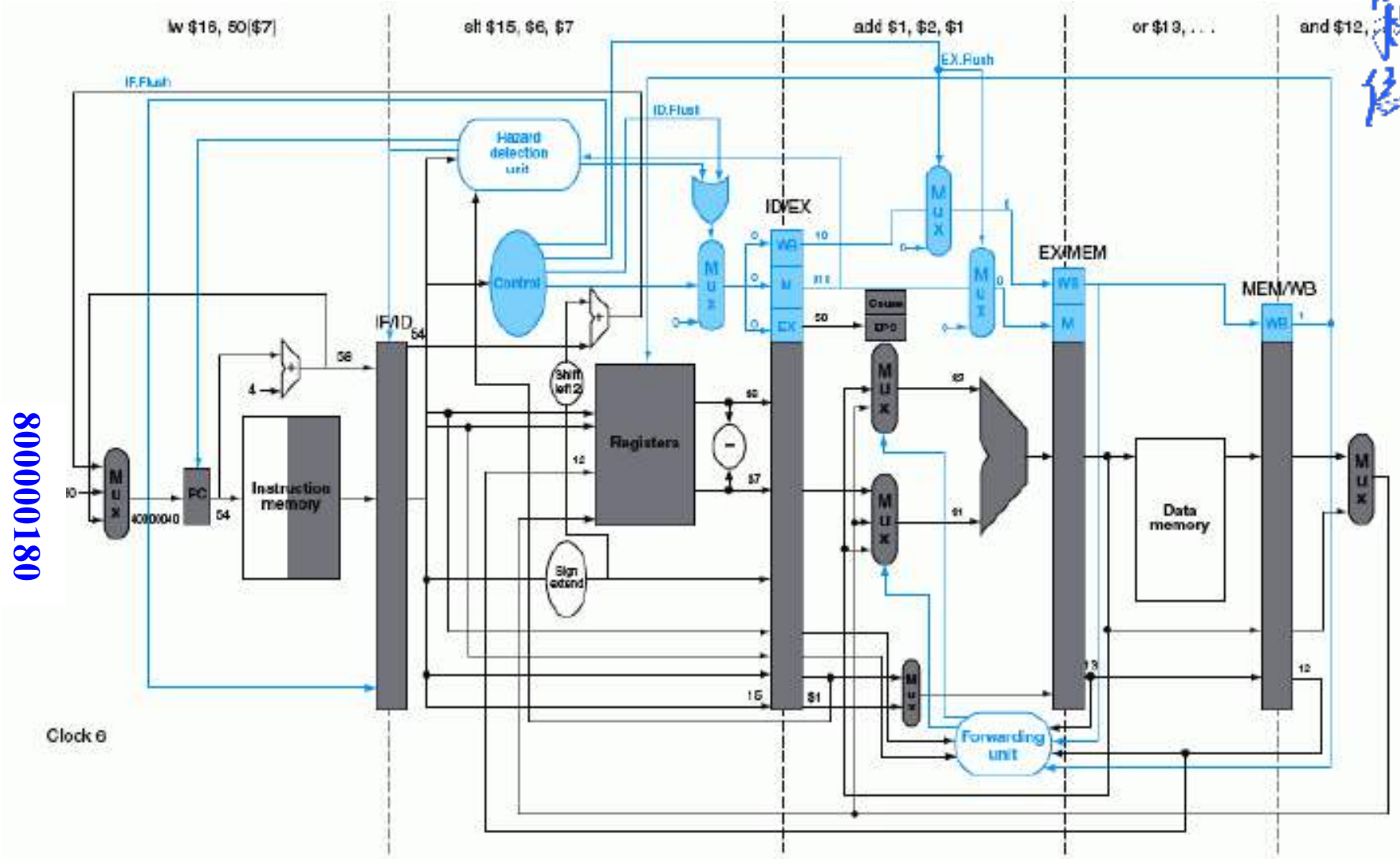
80000180_{hex}	sw	$25, 10000($0)
80000184_{hex}	sw	$26, 10004($0)

说明在指令add发生溢出异常时流水线的执行情况。

流水线计算机中的异常举例

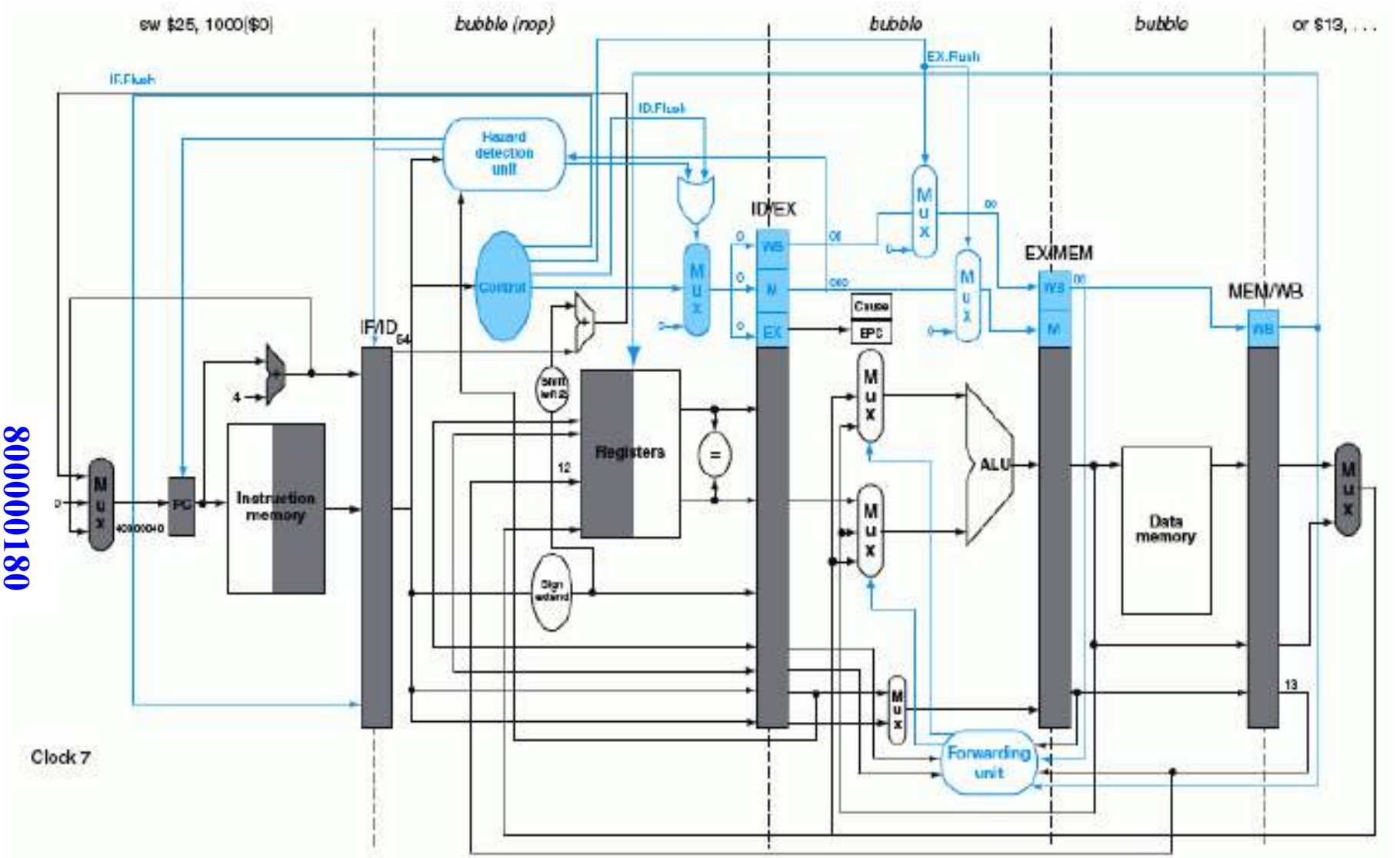
溢出在第六个时钟周期的EX阶段被检测到：

- 将add后面的指令地址($4C + 4 = 50_{hex}$)保存到EPC寄存器
- 溢出导致该周期后所有Flush信号置1，清除add的控制值(置0)



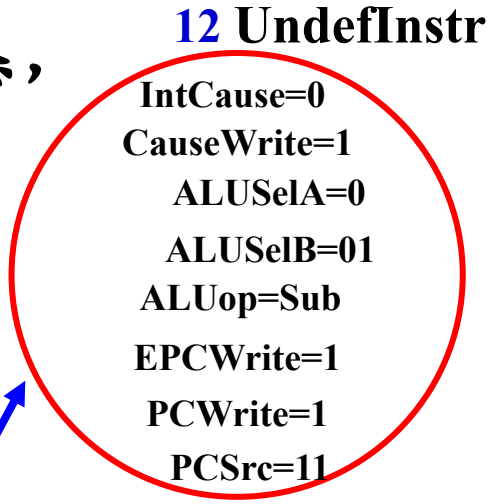
流水线计算机中的异常举例

- 第七个时钟周期将流水线中的指令转换成nop
- 开始从地址80000180_{hex}预取异常处理例程的第一条指令sw $25, 1000($0)

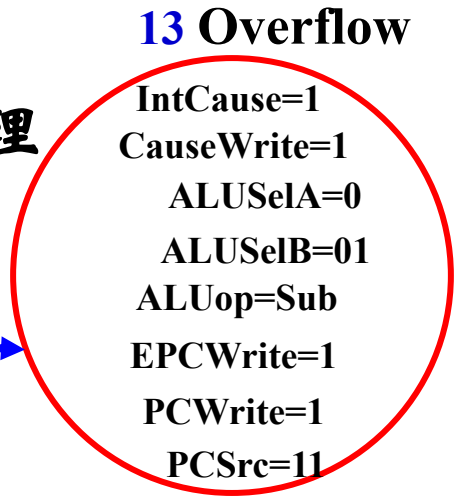


带异常处理的控制器设计

- 在有限状态机中增加异常处理的状态，每种异常占一个状态
- 异常处理状态中，考虑的基本控制
  - Cause寄存器的设置
  - 计算断点处的值，并送EPC
  - 将异常查询程序的入口地址送PC
  - 将中断允许位清0(关中断)
- 设要控制的数据通路中有两种异常处理
  - 未定义指令(Cause=0)：状态12
  - 数据溢出(Cause=1)：状态13



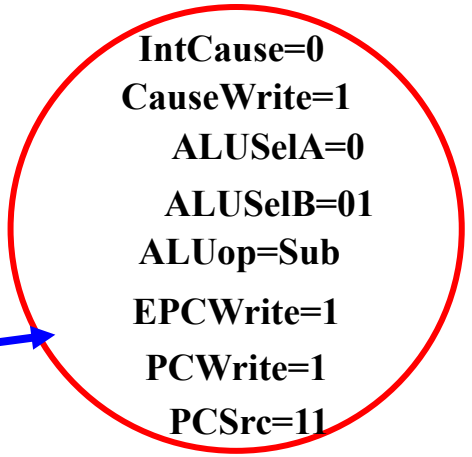
12 未定义指令异常状态



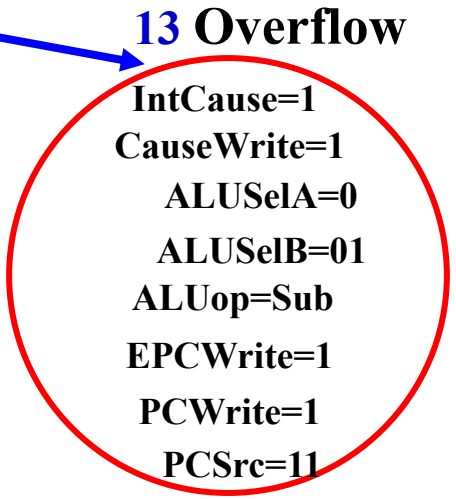
13 数据溢出异常状态

带异常处理的控制器设计

- 在有限状态机中增加异常处理的状态，每种异常占一个状态
- 异常处理状态中，考虑的基本控制
- 设要控制的数据通路中有两种异常处理
  - 未定义指令(Cause=0)：状态12
  - 数据溢出(Cause=1)：状态13
- 原状态转换图加两个异常处理状态
  - 如何检测是否发生这两种异常
    - 未定义指令：当指令译码器发现op字段是一个未定义的编码时
    - 数据溢出：当R-Type指令执行后在ALU输出端的Overflow为1时



12 未定义指令异常状态

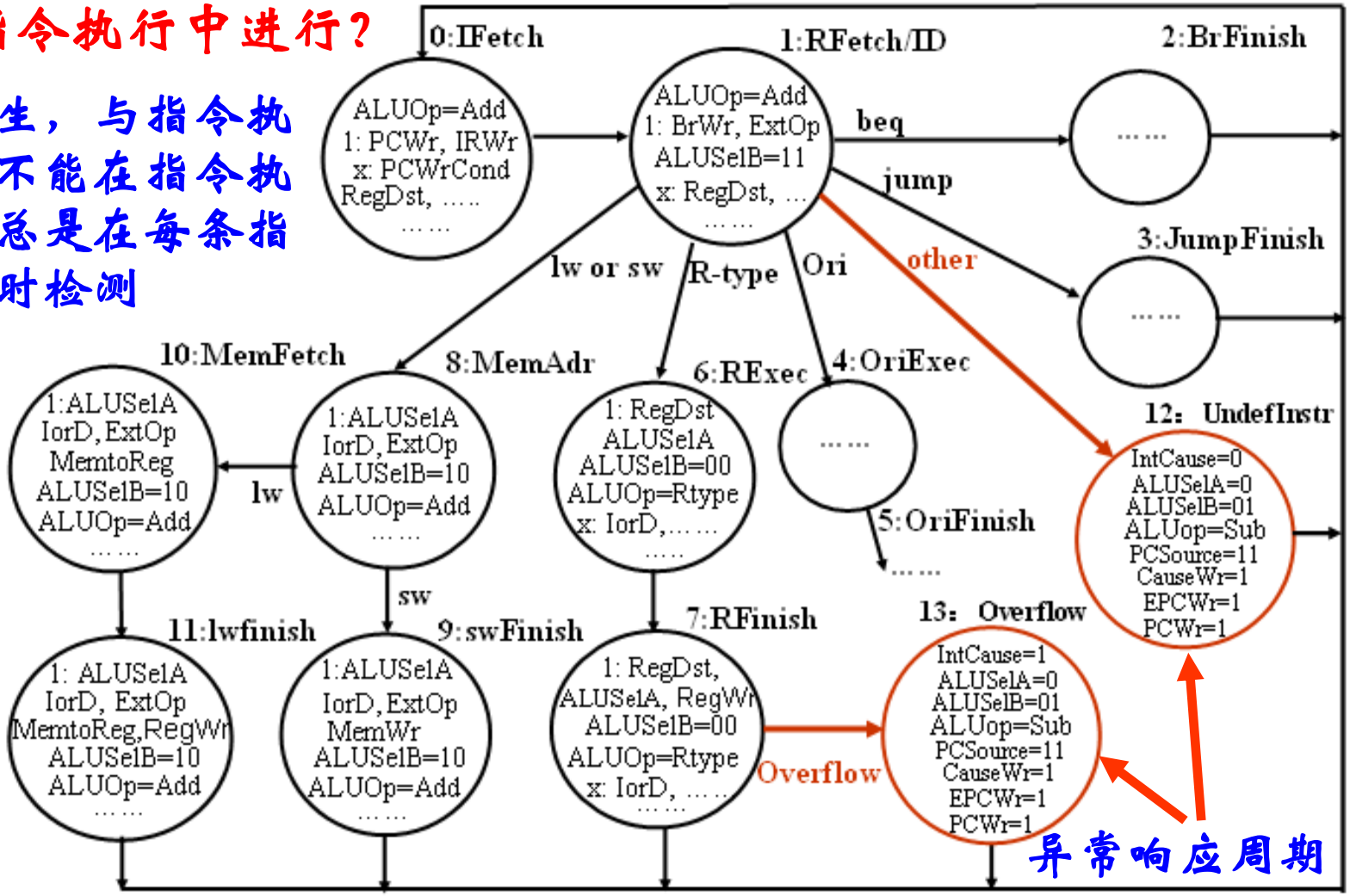


13 数据溢出异常状态

加入异常处理后的有限状态转换图

问题1：中断检测能否和异常检测一样在指令执行中进行？

中断随机发生，与指令执行不同步。不能在指令执行中检测，总是在每条指令执行结束时检测

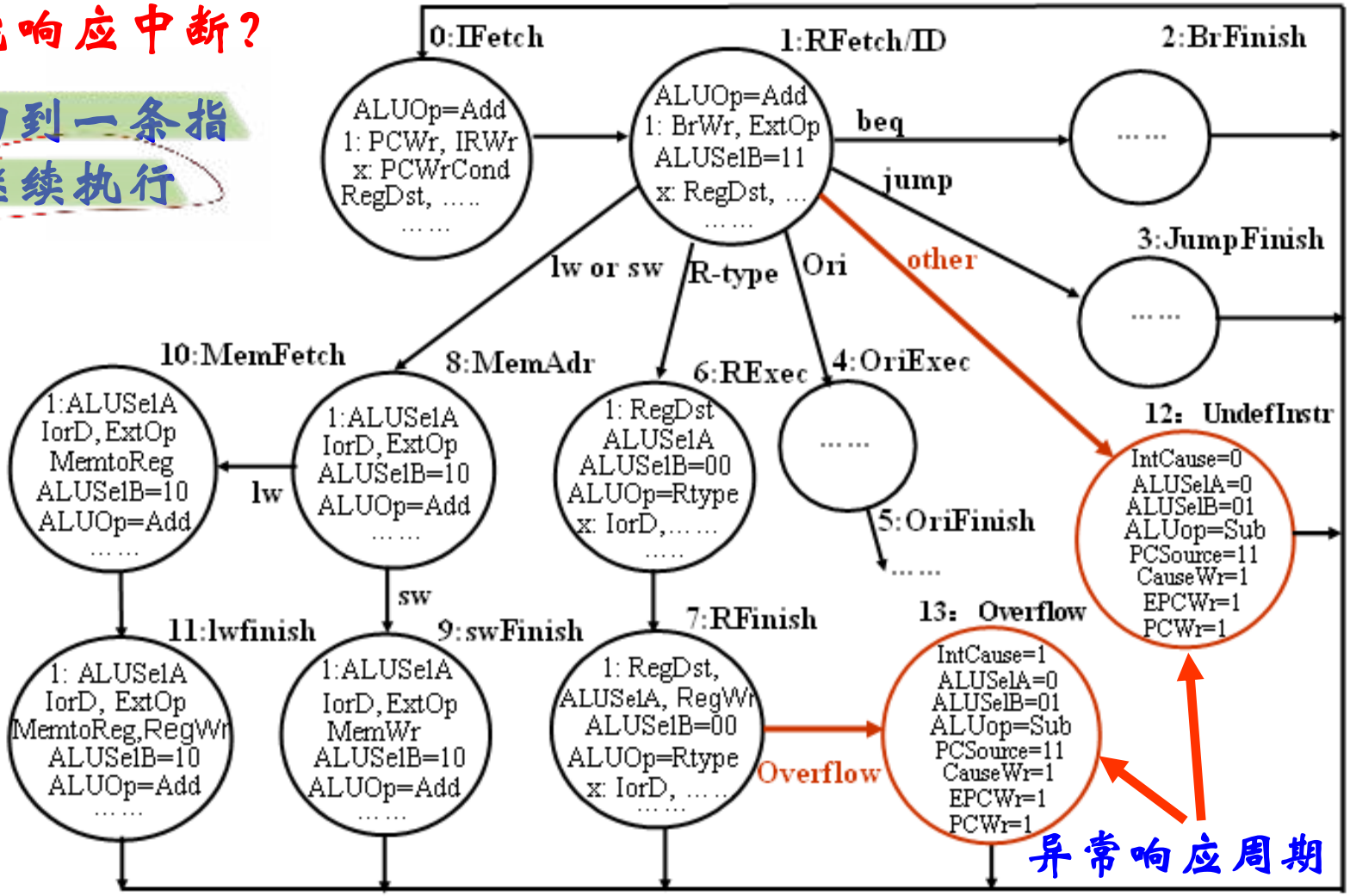


加入异常处理后的控制器设计可根据上述有限状态机实现！

加入异常处理后的有限状态转换图

问题2：为什么在指令执行中不能响应中断？

因为无法回到一条指令的中间继续执行



加入异常处理后的控制器设计可根据上述有限状态机实现！

4.10 指令级并行

回顾 - 提高处理器的性能



- 衡量CPU的性能指标：单位时间CPU完成的指令数

IPC：每个时钟周期内执行的指令数
CPI：每条指令的平均时钟周期数

处理器性能 = 主频 × IPC

处理器时间 = (程序指令数×CPI)/主频

➢ 性能提升主要手段

- 提升工作主频：KHz 增长到 GHz
  - 生产工艺不断进步带来的提升
  - 不断增加流水线级数
- 发掘并行处理能力
  - 指令级并行性/线程级并行性，等等

4.10.1 提高处理器指令级并行性的技术

□ 指令级并行

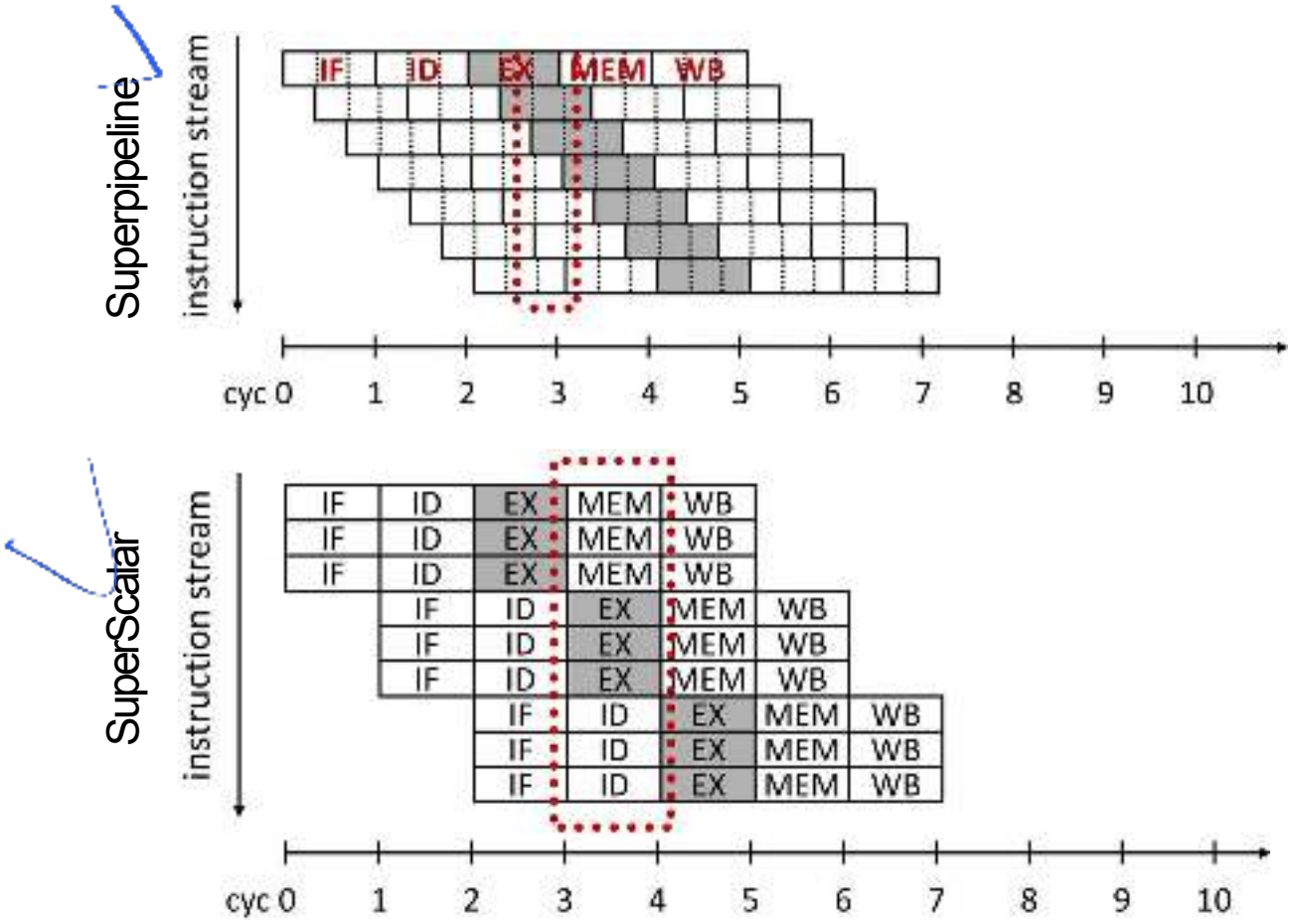
- 如果各指令之间不存在相关性，那么它们在流水线中是可以并行执行的，这种指令间潜在的重叠就是指令级并行(Instruction-Level Parallelism)
- 提高CPU执行速度
- ILP的方法：
  - 超流水线技术：增加流水线的深度以重叠更多的指令执行

CPU实现指令级并行可能存在的问题？如何解决？

型号	Pentium	P6架构	Pentium 4	Core
流水级数	5	12-13	20~31	14
主频MHz	66	200	2000~3600	2667

  - 多发射技术：一种单时钟周期内发射多条指令的机制。通过复制计算机内的部件数量，使得每个流水线可以启动多条指令执行

Superpipelined and SuperScalar Execution



Achieving full performance requires finding N "independent" instructions on every cycle

4.10.1 提高处理器指令级并行性的技术



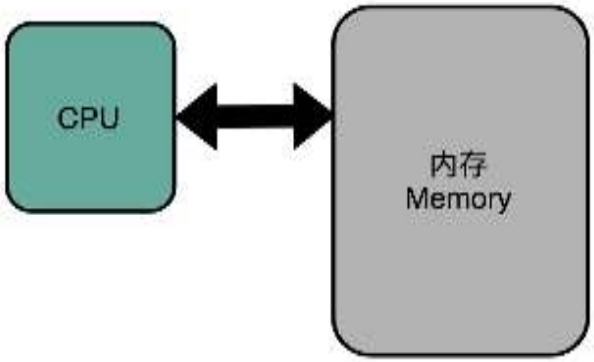
➢ 指令间存在的相关性

■ 结构相关/冒险（Structural Hazard）

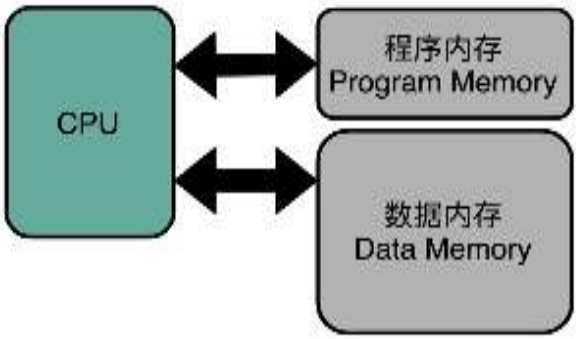
不同指令同时存取相同的寄存器或存储器，但这些指令之间不存在数据流。

例如：在冯·诺依曼存储结构中，数据和程序放在同一存储器。如果在某个时钟周期，指令 j 要向存储器写数据，而刚好指令 i 要从存储器取指令，这就出现了结构相关。

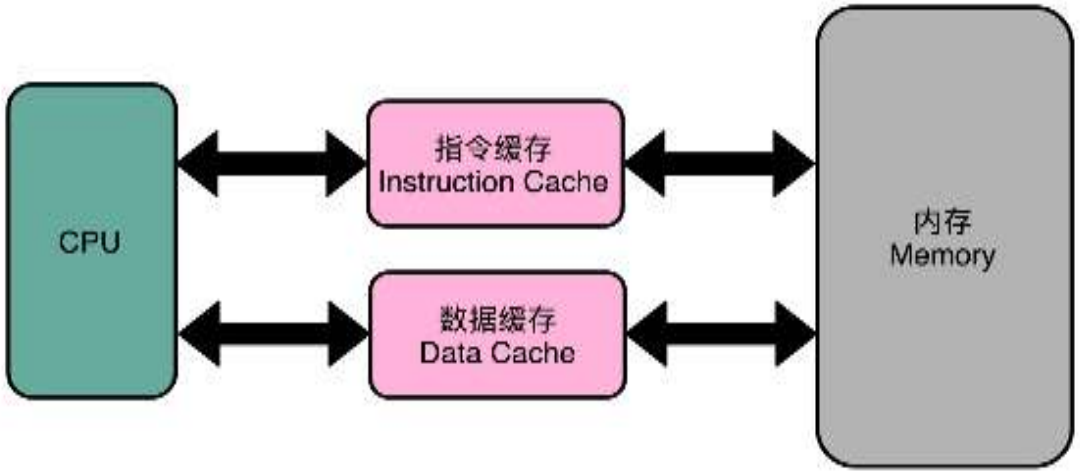
4.10.1 提高处理器指令级并行性的技术



冯·诺伊曼架构/普林斯顿架构
Princeton Archicture



哈佛架构
Harvard Architecture



现代CPU的混合架构
Hybrid Architecture

4.10.1 提高处理器指令级并行性的技术

➢ 指令间存在的相关性

■数据相关/冒险（Data Hazard）

某条指令的操作数依赖前一条或前几条指令的运行结果

① 写后读相关(RAW，Read After Write)，真相关

```
A=B+C
D=3*A   // 数据A的写后读
```

② 读后写相关(WAR，Write After Read)，反相关

```
A=B+C
B=D*2   // 数据B的读后写   （不能颠倒顺序）
```

③ 写后写相关(WAW，Write After Write)，输出相关

```
A=B+C
A=D*2   // 数据A的写后写   （不能颠倒顺序）
```

思考：C语言里的 i+=2 包含哪些数据相关？

4.10.1 提高处理器指令级并行性的技术



➢ 指令间存在的相关性

■数据相关/冒险（Data Hazard）

某条指令的操作数依赖前一条或前几条指令的运行结果

■① 写后读相关（RAW，Read After Write）

■② 读后写相关（WAR，Write After Read）

■③ 写后写相关（WAW，Write After Write）

C语言	汇编语言
i+=2	mov ax, i add ax, 2 mov i, ax

4.10.1 提高处理器指令级并行性的技术

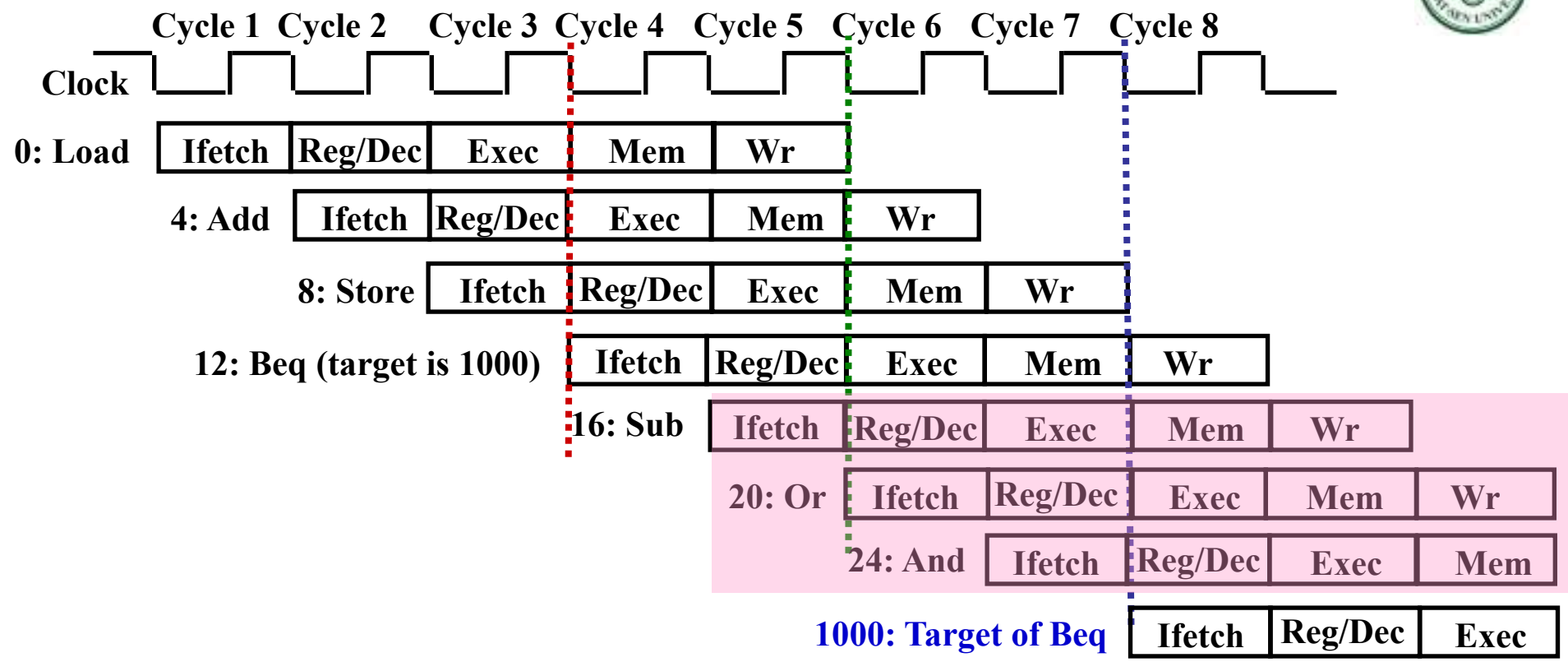


➢ 指令间存在的相关性

■ 控制相关/冒险（Control Hazard）

- 控制相关可看作是对程序计数器PC的RAW相关问题
- 取指阶段需读PC，而分支语句会在执行阶段计算出新的转移地址写入PC。当分支条件满足时，就会出现下条指令读PC(取指阶段)早于分支语句写该寄存器(执行阶段)的情况

回顾——MIPS五级流水线的控制相关性举例



➢Beq指令何时确定转移？转移目标地址在第几周期计算出来？

- 第六周期得到条件结果和转移地址，第七周期控制转移地址写入到PC，第八周期开始才能根据转移地址取指令
- 如果Beq指令执行结果是需要转移，流水线错误取出了Beq指令之后的3条指令

4.10.1 提高处理器指令级并行性的技术

- 实现多发射处理器的两种方法
  - 静态多发射：在执行前，由编译器帮助封装多条指令并处理冒险
    - 发射包：可以在给定时钟周期内发射多条指令
    - 发射槽：在给定时钟周期内能够发射指令的位置
    - 超长指令字：一类可以同时启动多个操作的指令集

4.10.1 提高处理器指令级并行性的技术



超长指令字(Very Long Instruction Word，VLIW)处理器

- 美国Yale大学Fisher首先提出
- 20世纪80年代，美国Multiflow和Cydrome公司设计的体系结构
- 一条指令来实现多个操作的并行执行
  - 一条指令多达上百位，有若干操作数，可以做不同的几种运算
  - 编译器选择可以并行执行的指令，并处理数据相关性
  - 硬件结构简单
  - 可以减少访存

4.10.1 提高处理器指令级并行性的技术



- 实现多发射处理器的两种方法
  - 静态多发射：在执行前，由编译器帮助封装多条指令并处理冒险
    - 发射包：可以在给定时钟周期内发射多条指令
    - 发射槽：在给定时钟周期内能够发射指令的位置
    - 超长指令字：一类可以同时启动多个操作的指令集
  - 动态多发射：在运行时，由处理器发射多条指令并处理冒险
    - 超标量：一种高级流水线技术，可以使每个周期处理器能执行的指令数超过一条
    - 动态流水线调度：对指令进行重新排序以避免阻塞的硬件支持

4.10.1 提高处理器指令级并行性的技术

超标量(Superscalar)处理器

- 1987年提出，为改善标量指令执行性能而设计
- 试图在一个周期取出多条指令并行执行，通过在处理器中内置多条流水线来同时执行多个处理
- 本质是以空间换取时间，在不同流水线中不相关地执行多条指令。允许指令以不同于原程序顺序的次序执行
- 超标量技术提高处理器的IPC(instructions per cycle)
- 超标量流水线中，并行执行的流水线数称为超标度
  - 从Pentium到Ivy Bridge处理器的超标度为2~4

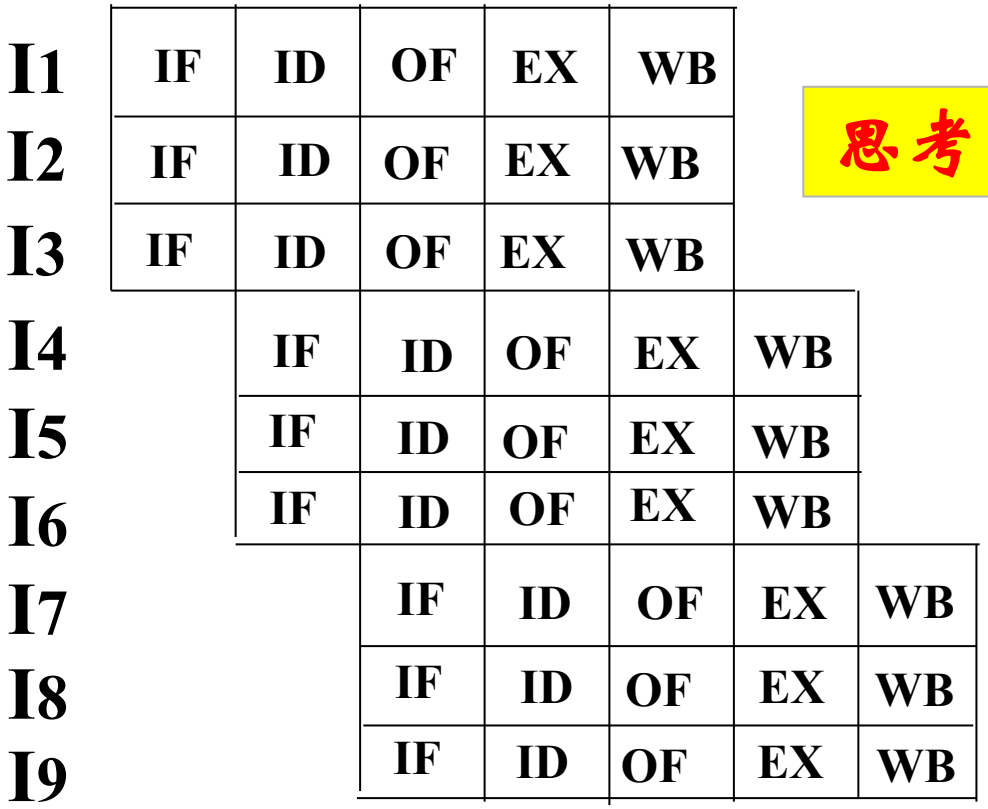
➢标量处理器(SISD)：在同一时间内只执行一条指令且处理一个数据(整数或者浮点数)

➢向量处理器(SIMD)：在同一时间内执行一条指令但处理多个数据

4.10.1 提高处理器指令级并行性的技术

超标量处理器

■在超标量流水线中，并行执行的流水线条数称之为超标度



思考：超标度为几？

4.10.1 提高处理器指令级并行性的技术



2．超标量技术

```
Load   R1←R2          |    Add    R3←R3,"1"
Add    R3←R3,"1"      |    Add    R4←R4,R2
Add    R4←R4,R2       |    Store  [R4]←R0
```

指令独立，可以并行执行　　不能并行执行

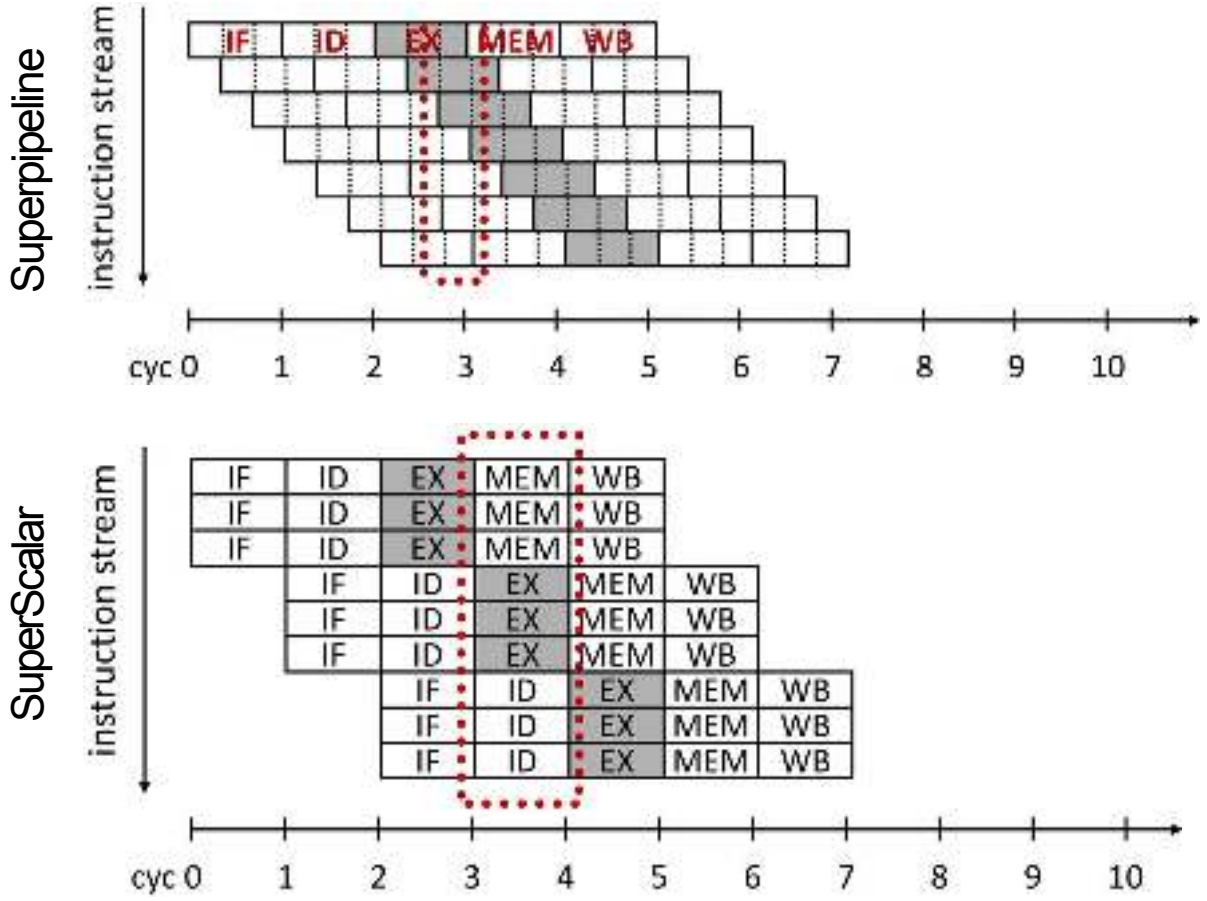
4.10.1 提高处理器指令级并行性的技术



向量处理器

- 标量处理器(SISD)：在同一时间内只执行一条指令且处理一个数据(整数或者浮点数)
- 向量处理器(SIMD)：在同一时间内执行一条指令但处理多个数据

Superpipelined and SuperScalar Execution



Achieving full performance requires finding N "independent" instructions on every cycle

Increasing parallelism via dataflow

- **Parallelism limited by many *false dependencies*, particularly *sequential program order***

- **Dataflow tracks how instructions actually depend on each other**
  - ***True dependence*: read-after-write**
  - ***False dependence:* write-after-write, write-after-read**

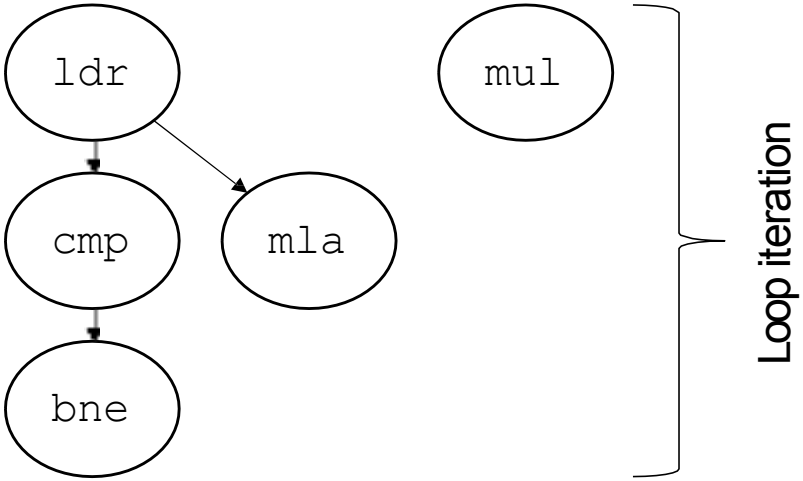
Dataflow increases parallelism by eliminating unnecessary dependences

Example: Dataflow in polynomial evaluation

```
ldr     r5, [r3], #4
cmp     r1, r3
mla     r0, r4, r5, r0
mul     r4, r2, r4
bne     .L3

ldr     r5, [r3], #4
cmp     r1, r3
mla     r0, r4, r5, r0
mul     r4, r2, r4
bne     .L3

...
```

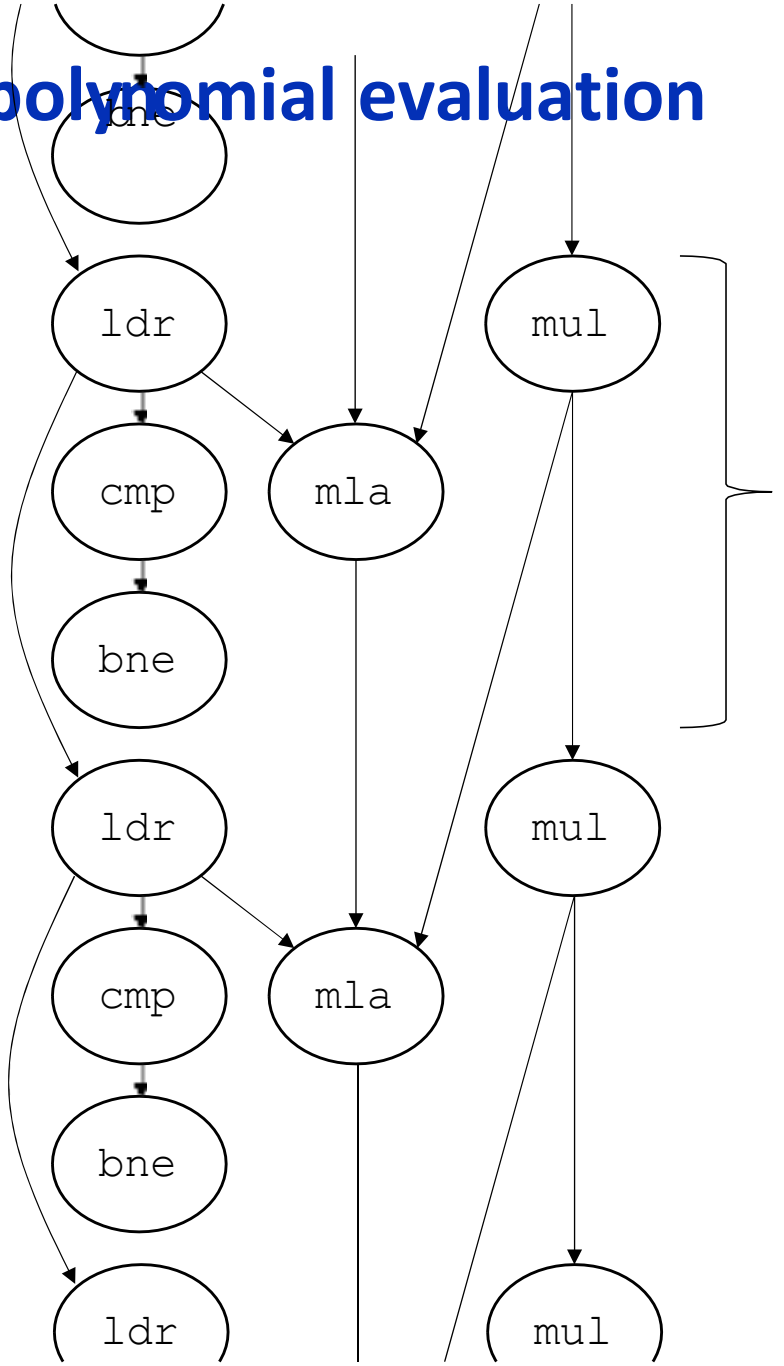


Example: Dataflow in polynomial evaluation

```
ldr     r5, [r3], #4
cmp     r1, r3
mla     r0, r4, r5, r0
mul     r4, r2, r4
bne     .L3

ldr     r5, [r3], #4
cmp     r1, r3
mla     r0, r4, r5, r0
mul     r4, r2, r4
bne     .L3

...
```

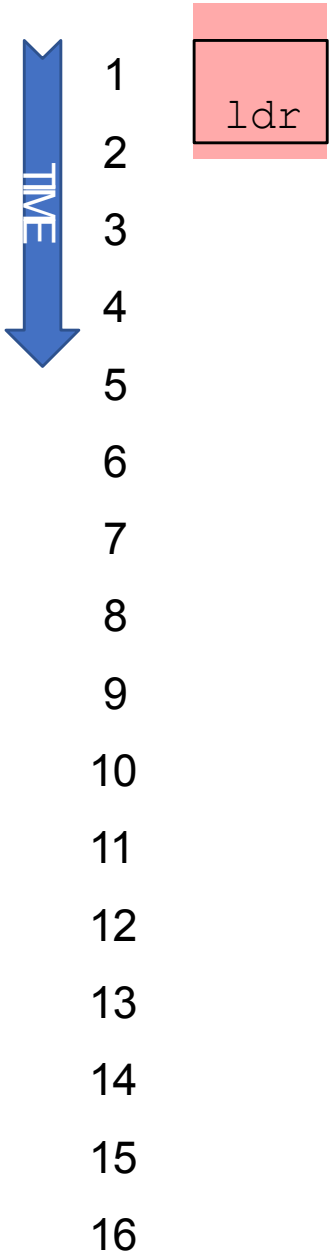


Example: Dataflow polynomial execution

- **Execution only, with perfect scheduling & unlimited execution units**
  - `ldr`, `mul` **execute in 2 cycles**
  - `cmp`, `bne` **execute in 1 cycle**
  - `mla` **executes in 3 cycles**

- **Q: Does dataflow speedup execution? By how much?**

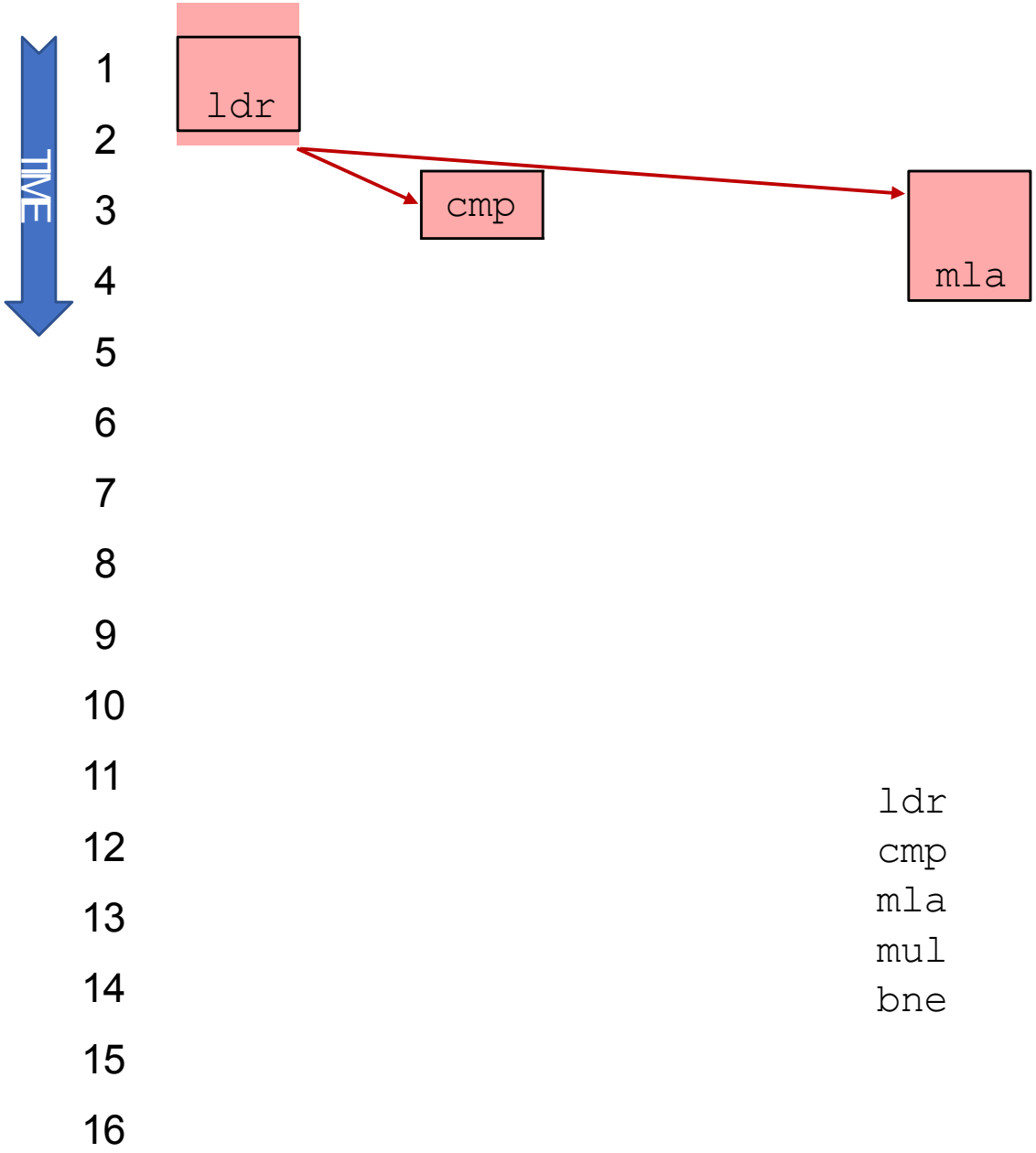
- **Q: What is the performance bottleneck?**



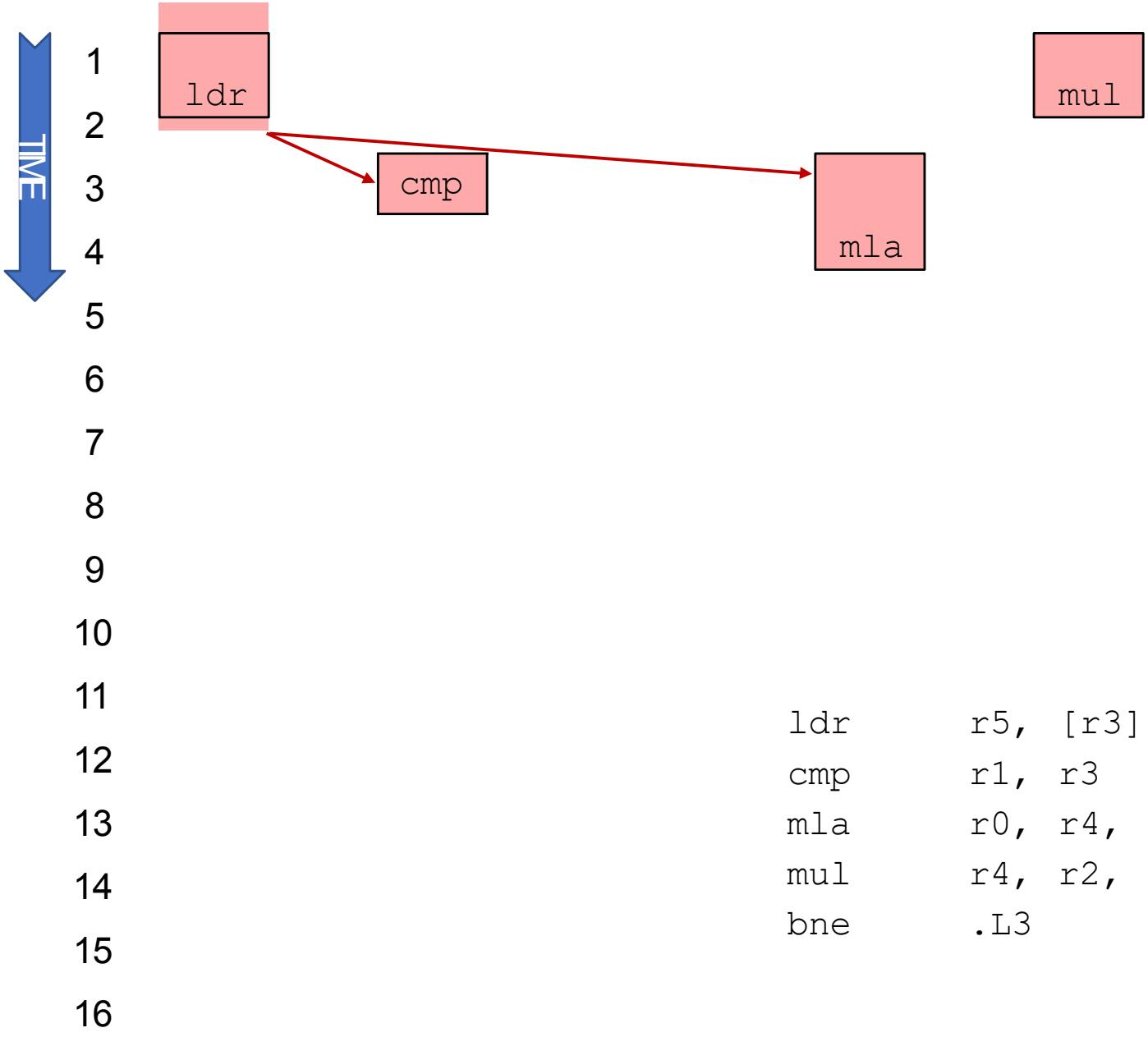

```
ldr     r5, [r3], #4
cmp     r1, r3
mla     r0, r4, r5, r0
mul     r4, r2, r4
bne     .L3
```



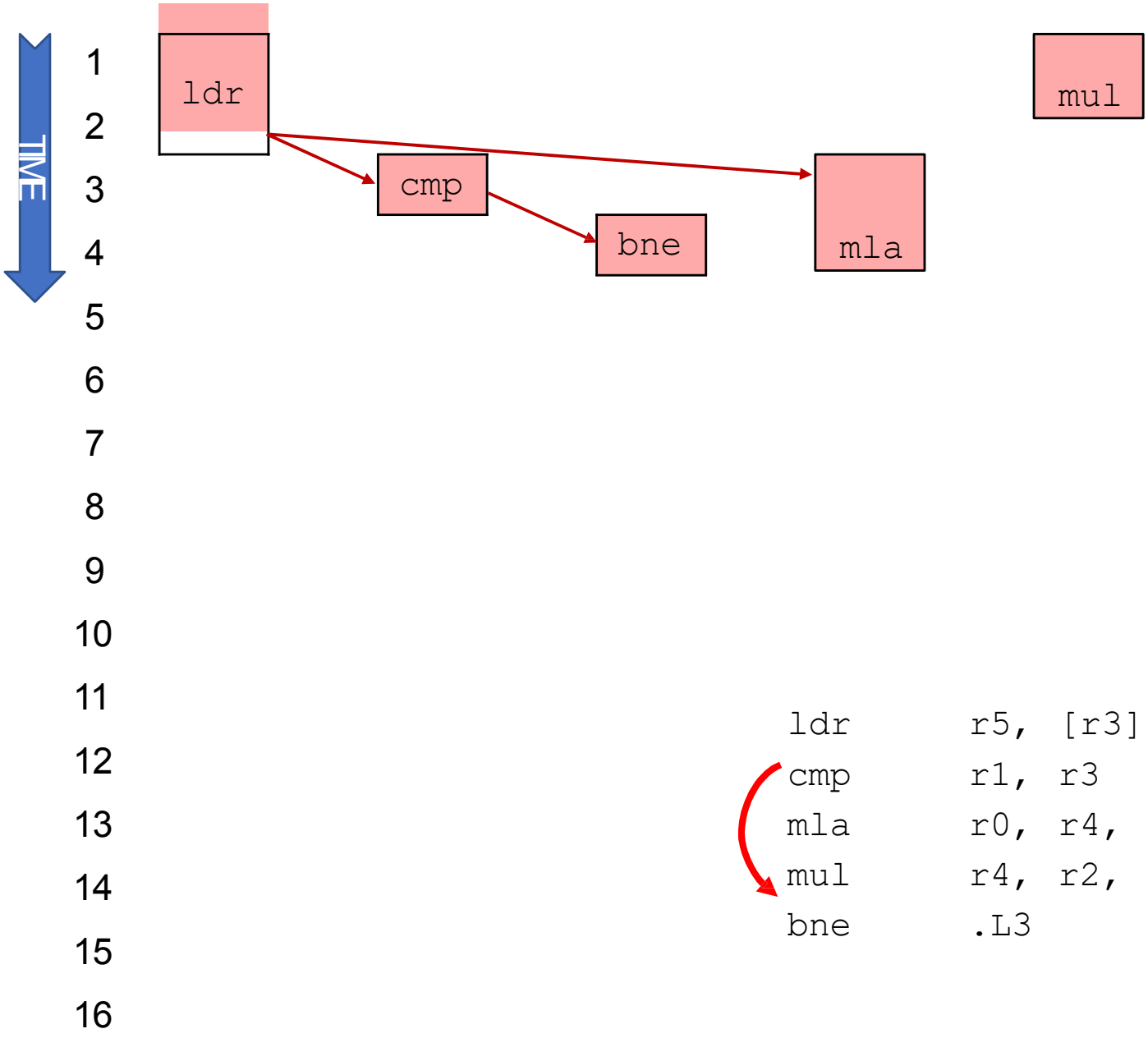

```
ldr     r5, [r3], #4
cmp     r1, r3
mla     r0, r4, r5, r0
mul     r4, r2, r4
bne     .L3
```



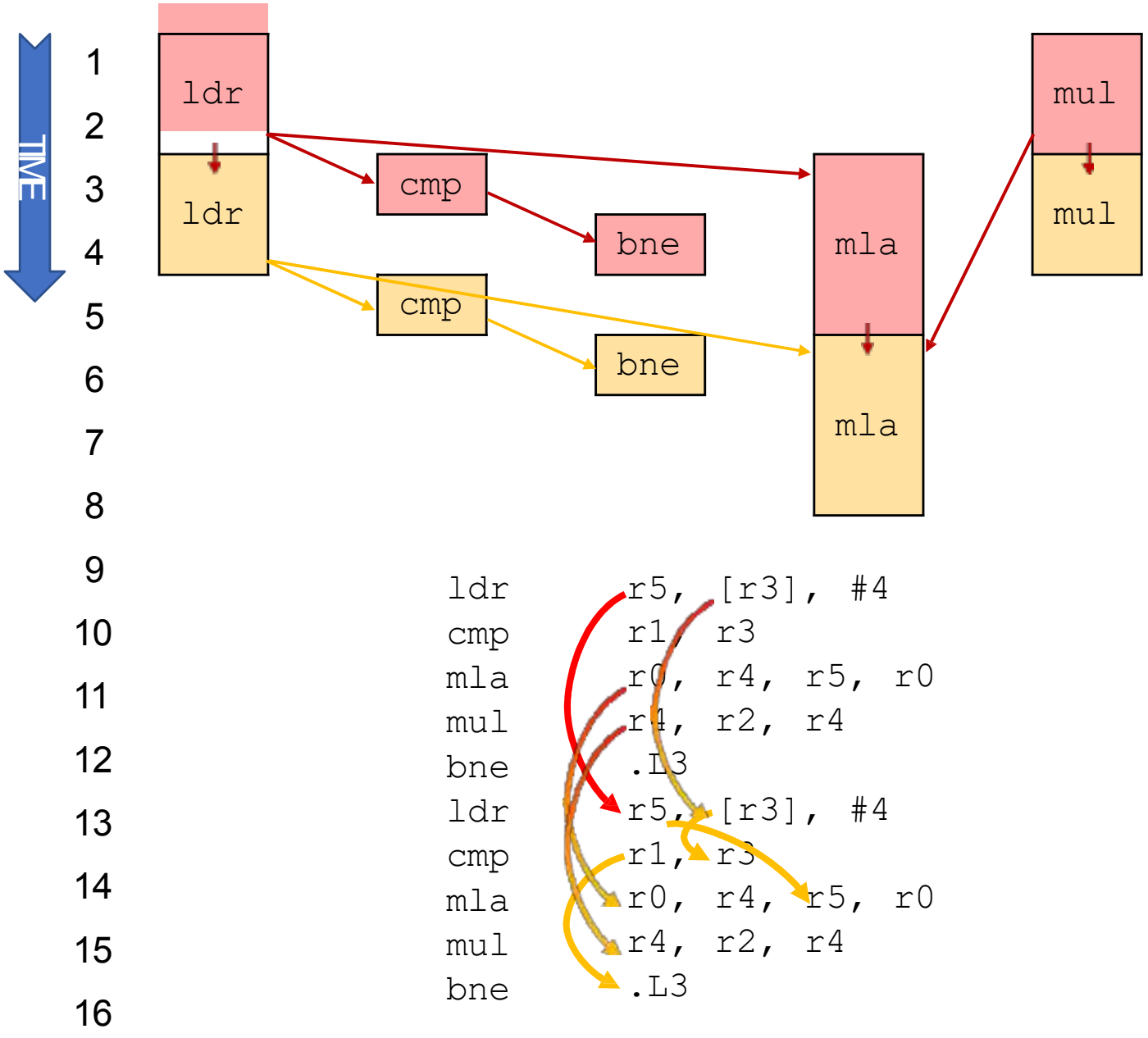

```
ldr     r5, [r3], #4
cmp     r1, r3
mla     r0, r4, r5, r0
mul     r4, r2, r4
bne     .L3
```

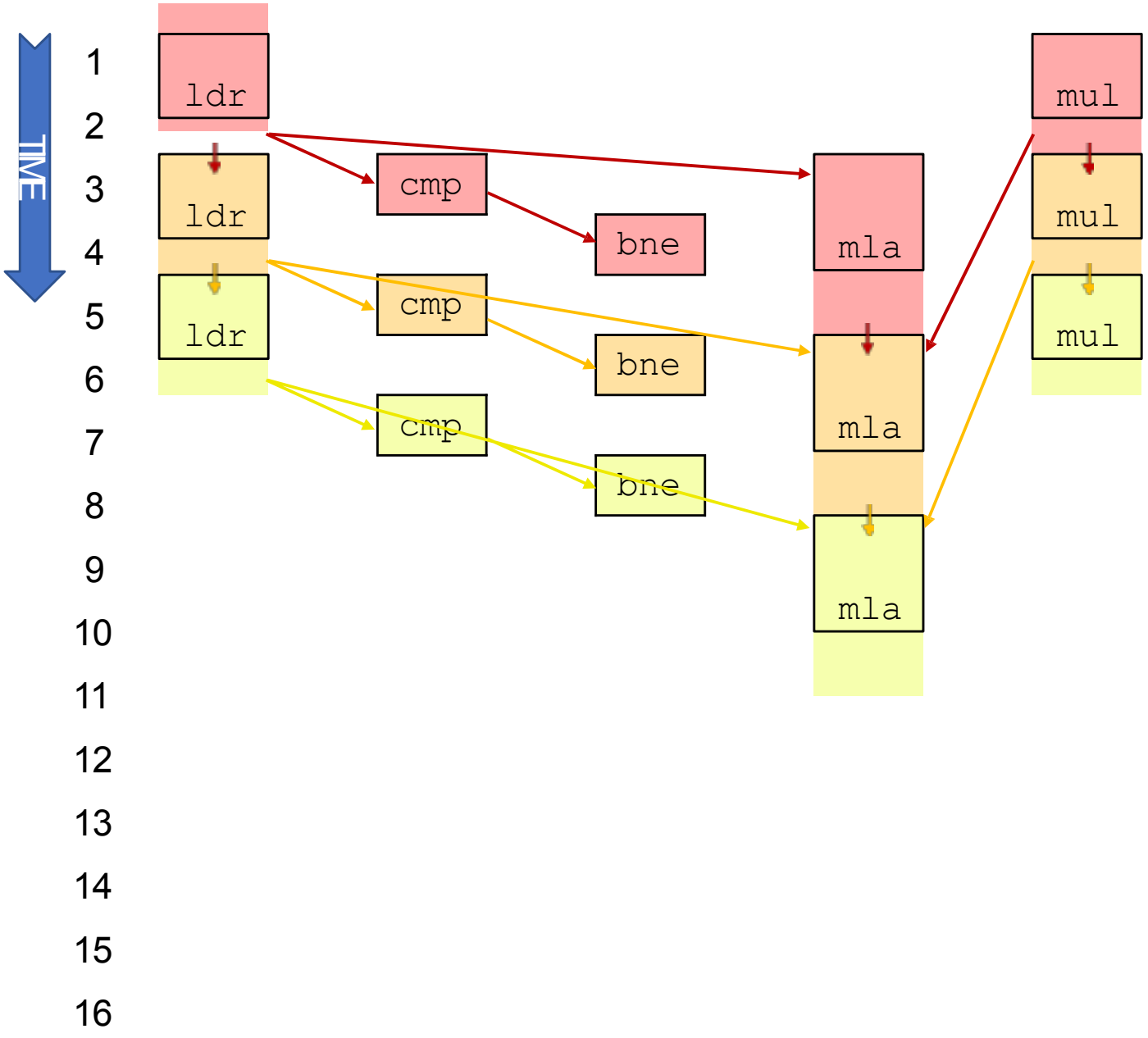


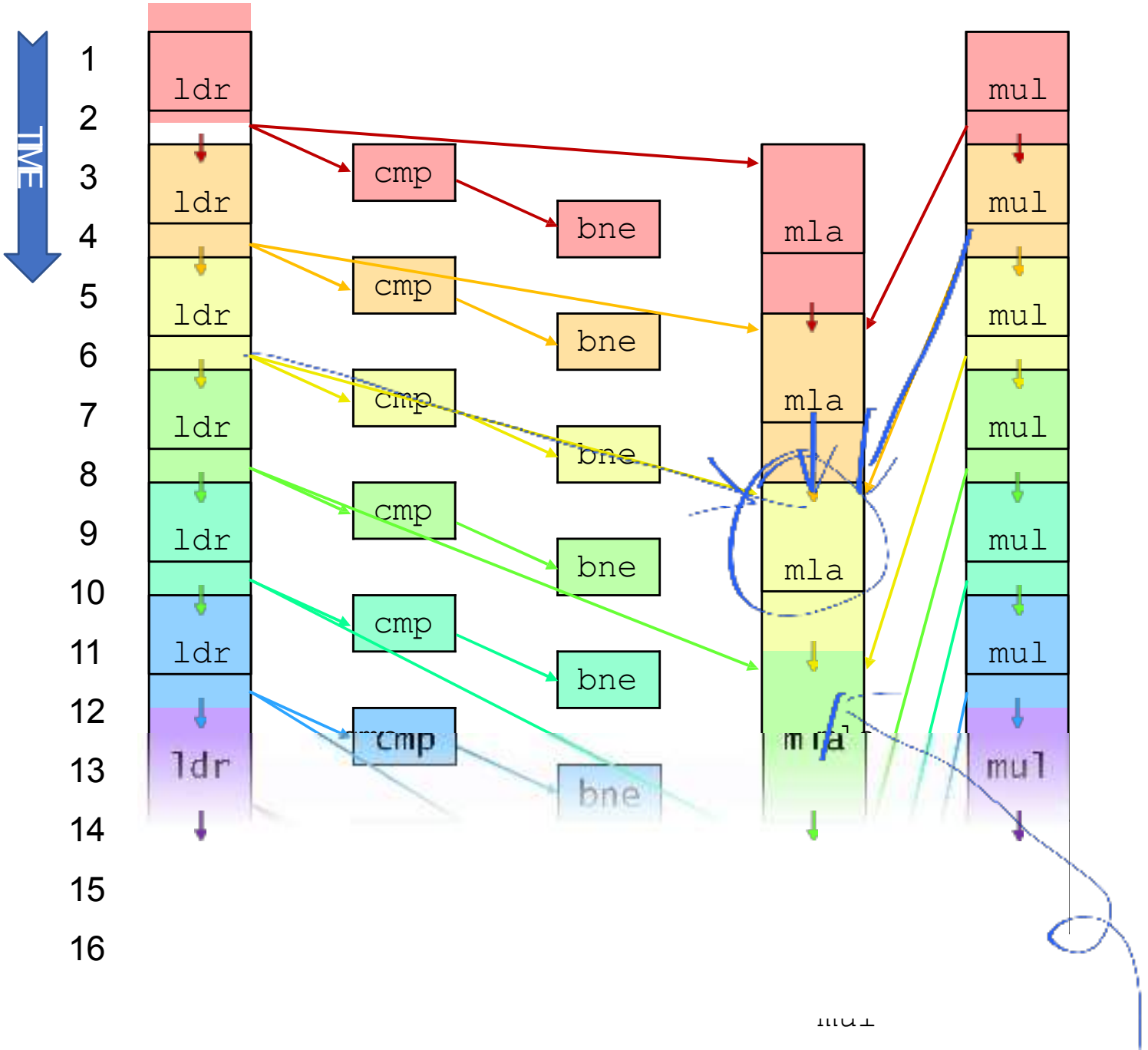

```
ldr     r5, [r3], #4
cmp     r1, r3
mla     r0, r4, r5, r0
mul     r4, r2, r4
bne     .L3
```




```
ldr     r5, [r3], #4
cmp     r1, r3
mla     r0, r4, r5, r0
mul     r4, r2, r4
bne     .L3
```


TIME
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
ldr
ldr
cmp
bne
cmp
bne
mla
mla
mul
mul
ldr r5, [r3], #4
cmp r1, r3
mla r0, r4, r5, r0
mul r4, r2, r4
bne .L3
ldr r5, [r3], #4
cmp r1, r3
mla r0, r4, r5, r0
mul r4, r2, r4
bne .L3


TIME
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
ldr
ldr
ldr
cmp
cmp
cmp
bne
bne
bne
mla
mla
mla
mul
mul
mul


TIME
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
ldr
cmp
bne
mla
mul

Example: Dataflow polynomial execution

- Q: Does dataflow speedup execution? By how much?
  - Yes! 3 cycles / loop iteration
  - Instructions per cycle (IPC) = 5/3≈1.67 (vs. 1 for perfect pipelining)
- Q: What is the performance bottleneck?
  - `mla`: Each `mla` depends on previous `mla` & takes 3 cycles
  - This program is latency-bound

Latency Bound

- What is the "critical path" of the computation?
  - Longest path across iterations in dataflow graph
  - E.g., `mla` in last slide (but could be multiple ops)

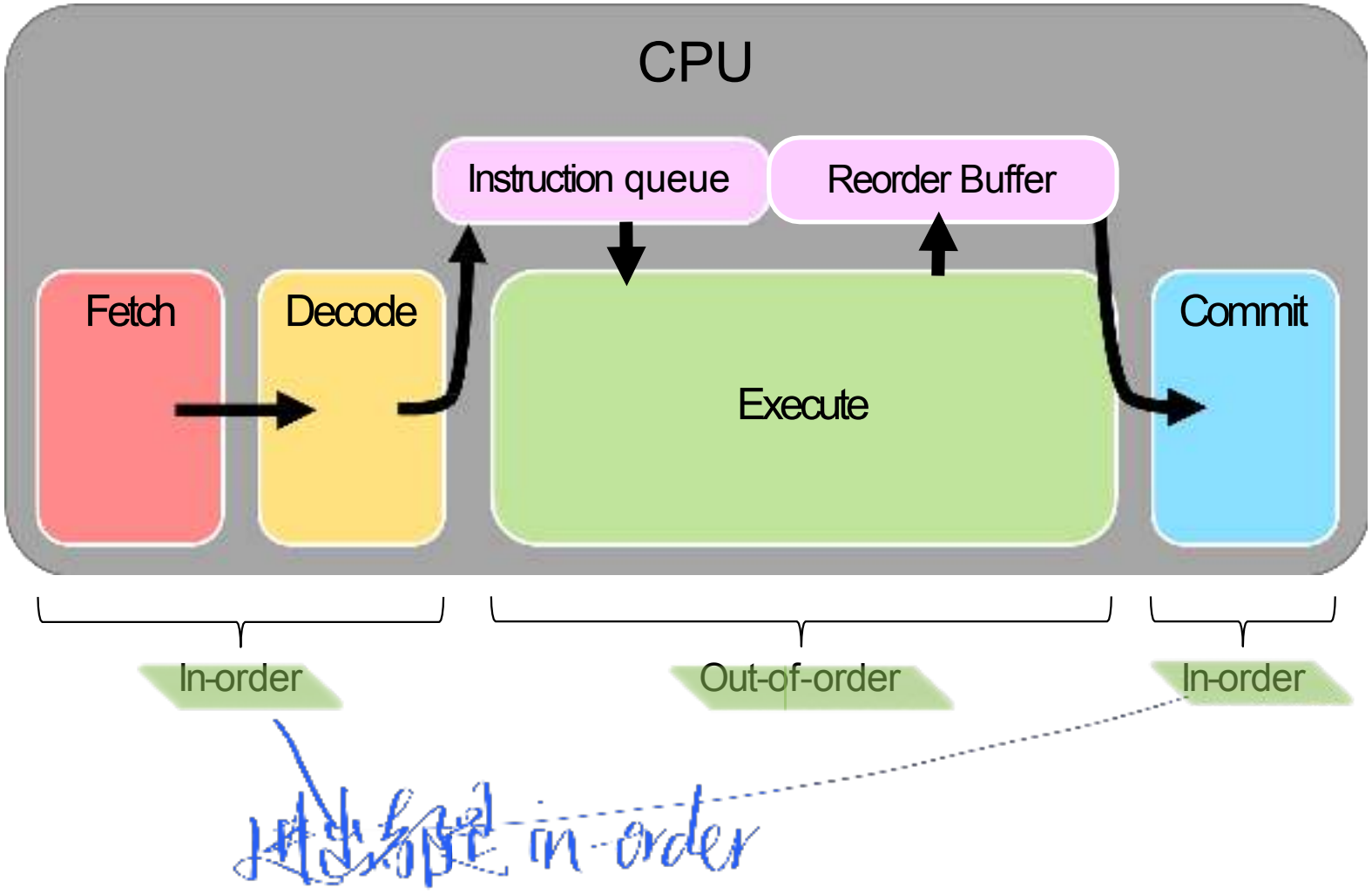
- Critical path limits maximum performance
- Real CPUs may not achieve latency bound, but useful mental model + tool for program analysis

Out-of-order (OoO) execution uses dataflow to increase parallelism

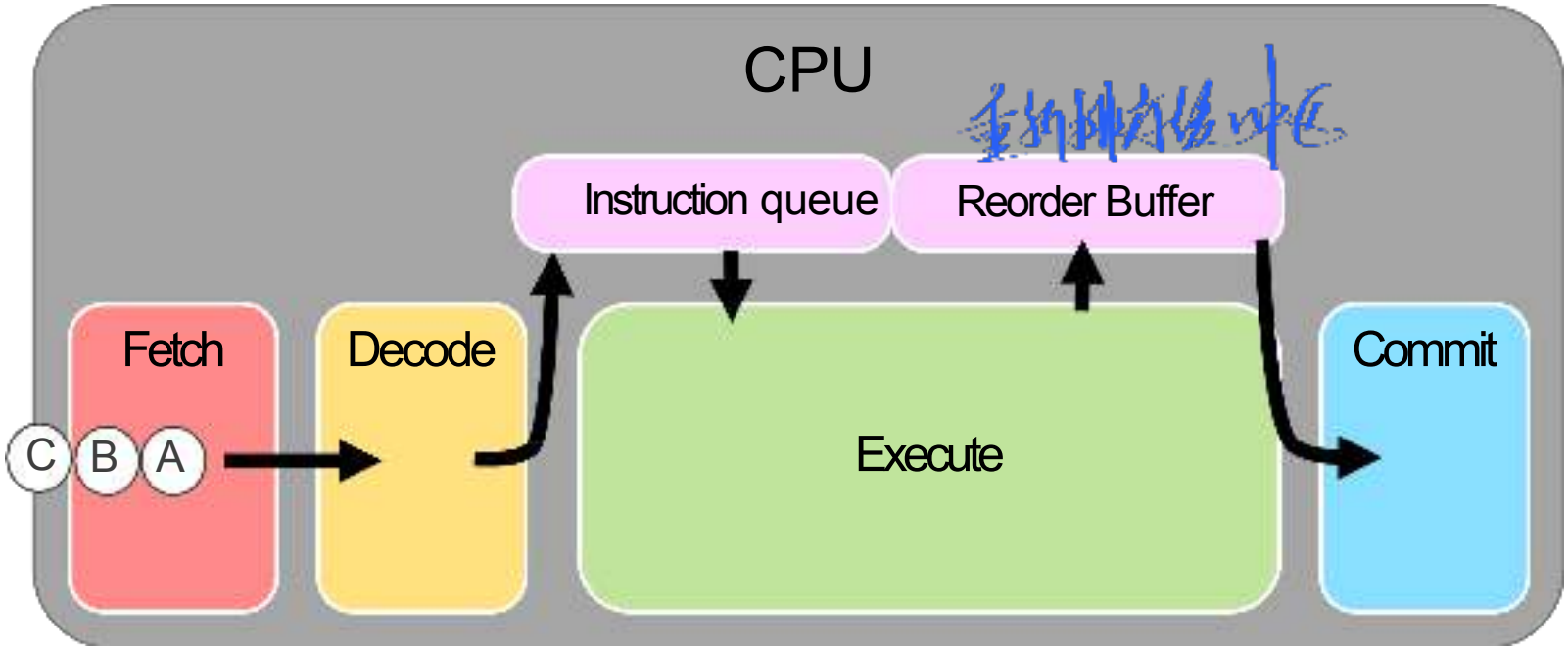
- Idea: Execute programs in dataflow order, but give the *illusion* of sequential execution

- This is a "restricted dataflow" model
  - *Restricted* to instructions near those currently committing
  - (Pure dataflow processors also exist that expose dataflow to software)

High-level OoO microarchitecture

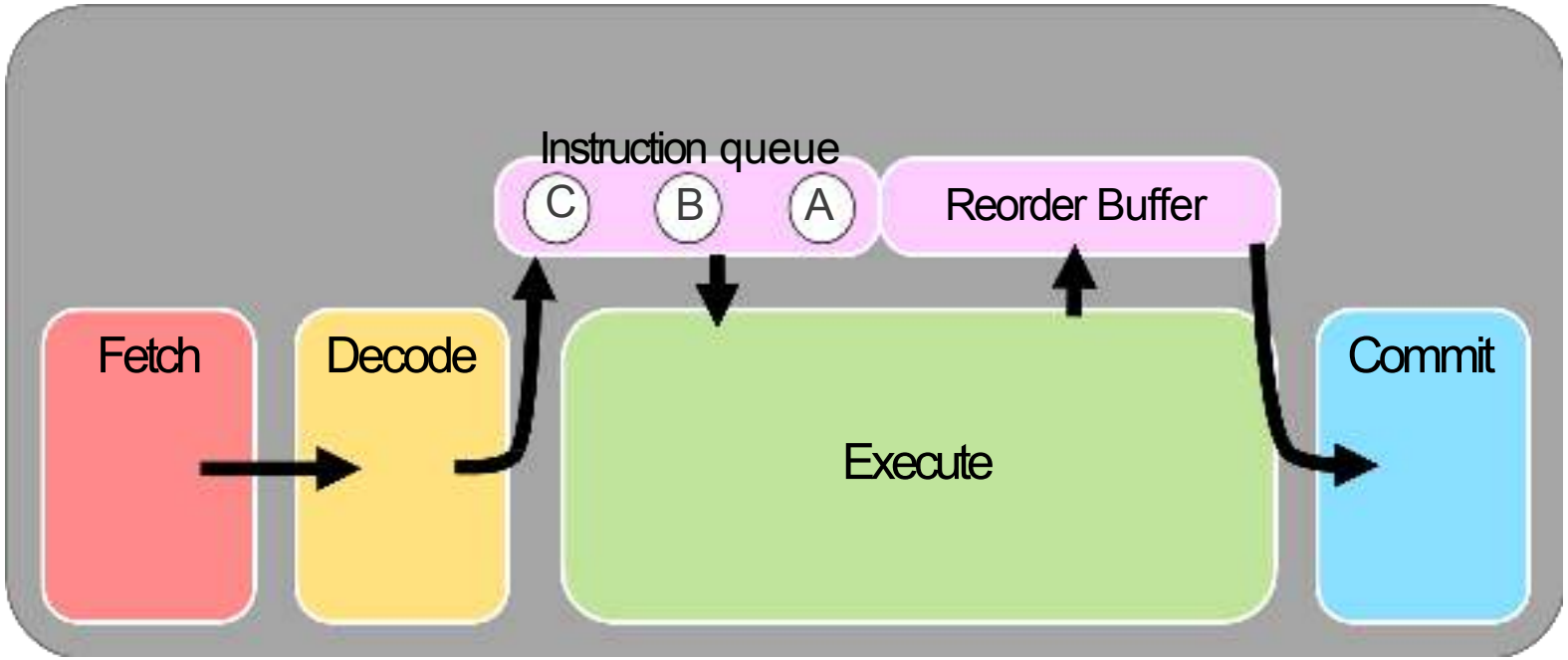


OoO is hidden behind in-order frontend & commit



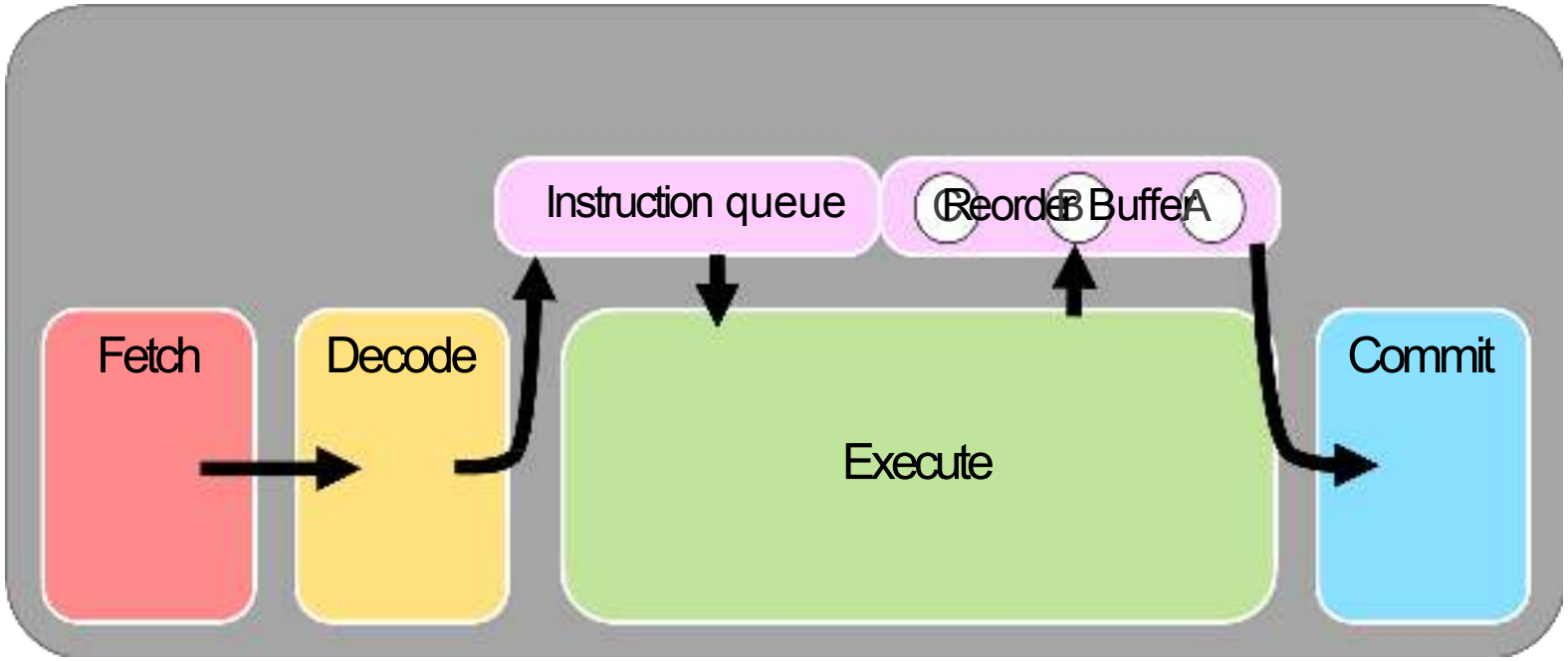
- Instructions only enter instruction queue(IQ) and leave reorder buffer(ROB) in program order; all bets are off in between!

OoO is hidden behind in-order frontend & commit



- **Instructions only enter instruction queue(IQ) and leave reorder buffer(ROB) in program order; all bets are off in between!**

OoO is hidden behind in-order frontend & commit



- Instructions only enter instruction queue(IQ) and leave reorder buffer(ROB) in program order; all bets are off in between!

Example: OoO polynomial evaluation

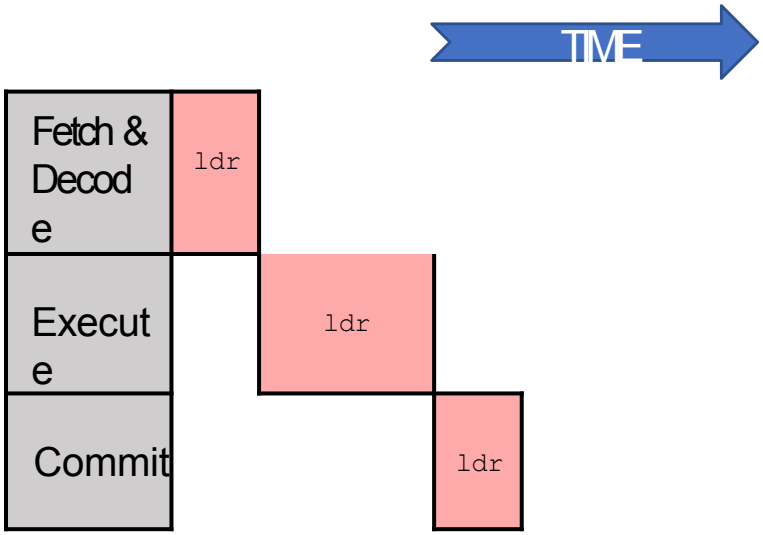
- Q: Does OoO speedup execution? By how much?
- Q: What is the performance bottleneck?
- Assume perfect forwarding & branch prediction

Example: OoO polynomial evaluation pipeline diagram

`ldr`, `mul` execute in 2 cycles

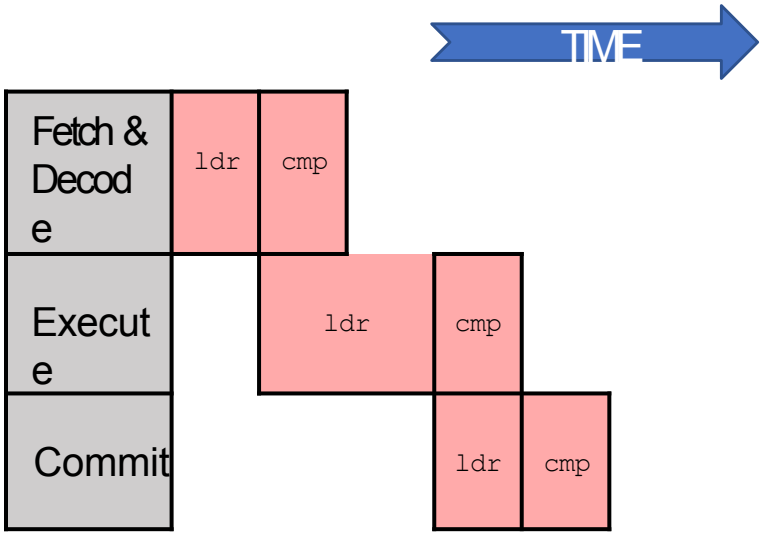
`cmp`, `bne` execute in 1 cycle

`mla` executes in 3 cycles



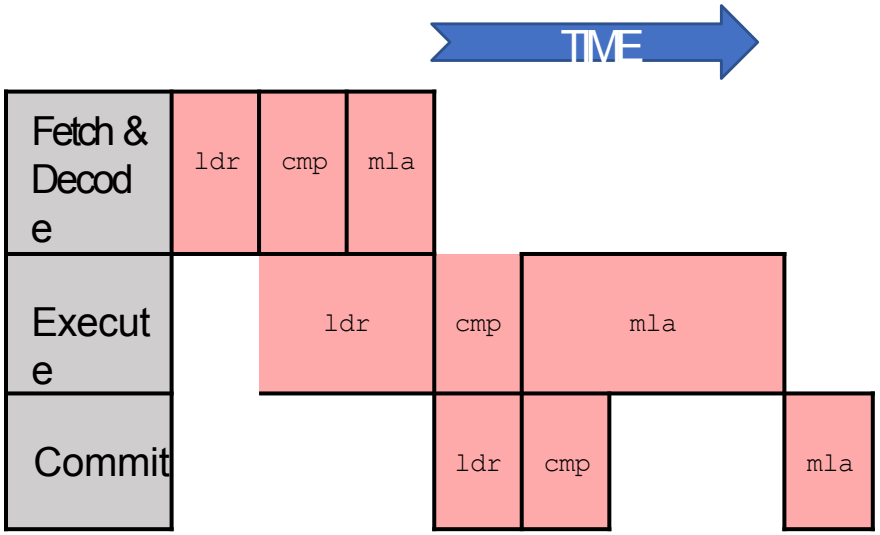
Example: OoO polynomial evaluation pipeline diagram

`ldr`, `mul` execute in 2 cycles
`cmp`, `bne` execute in 1 cycle
`mla` executes in 3 cycles



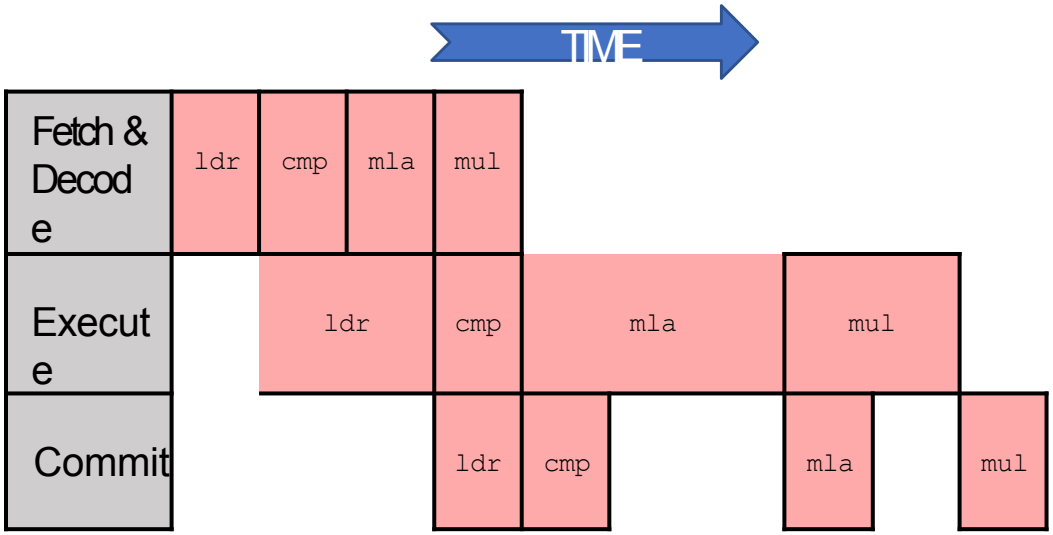
Example: OoO polynomial evaluation pipeline diagram

`ldr`, `mul` execute in 2 cycles
`cmp`, `bne` execute in 1 cycle
`mla` executes in 3 cycles



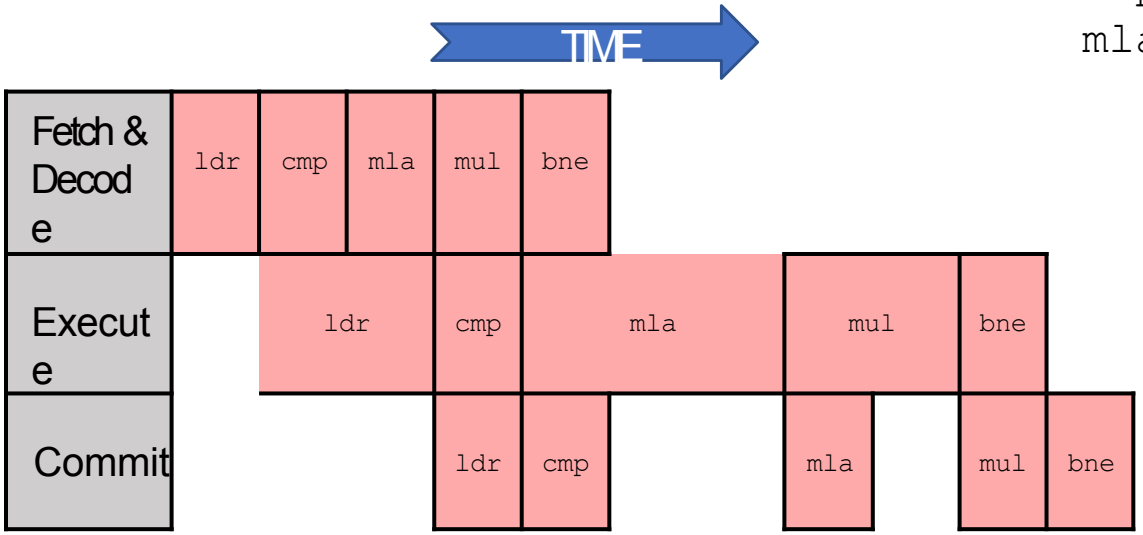
Example: OoO polynomial evaluation pipeline diagram

`ldr`, `mul` execute in 2 cycles
`cmp`, `bne` execute in 1 cycle
`mla` executes in 3 cycles



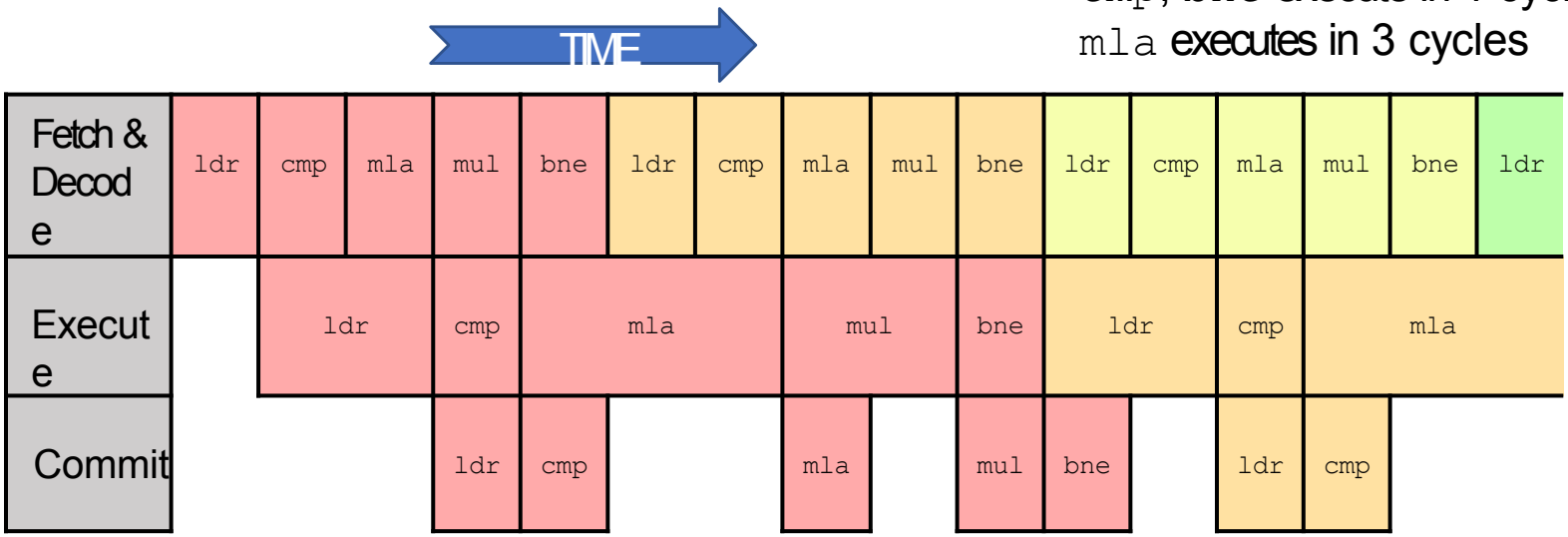
Example: OoO polynomial evaluation pipeline diagram

`ldr`, `mul` execute in 2 cycles
`cmp`, `bne` execute in 1 cycle
`mla` executes in 3 cycles



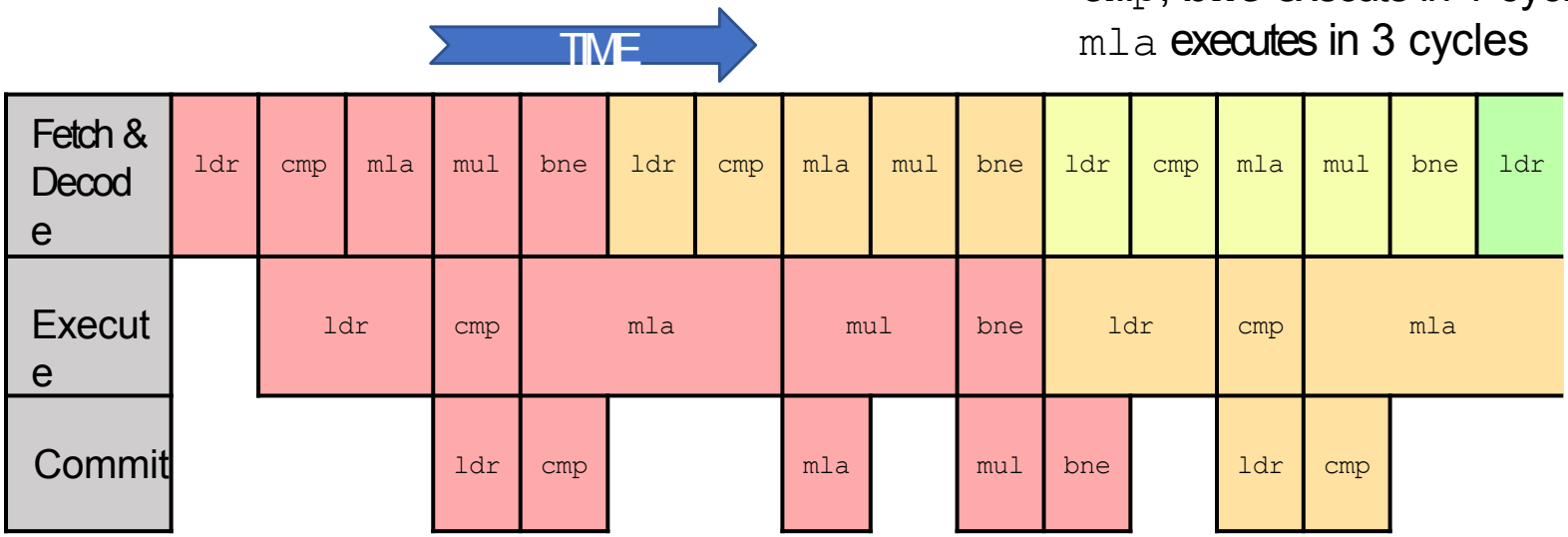
Example: OoO polynomial evaluation pipeline diagram

`ldr`, `mul` execute in 2 cycles
`cmp`, `bne` execute in 1 cycle
`mla` executes in 3 cycles



Example: OoO polynomial evaluation pipeline diagram

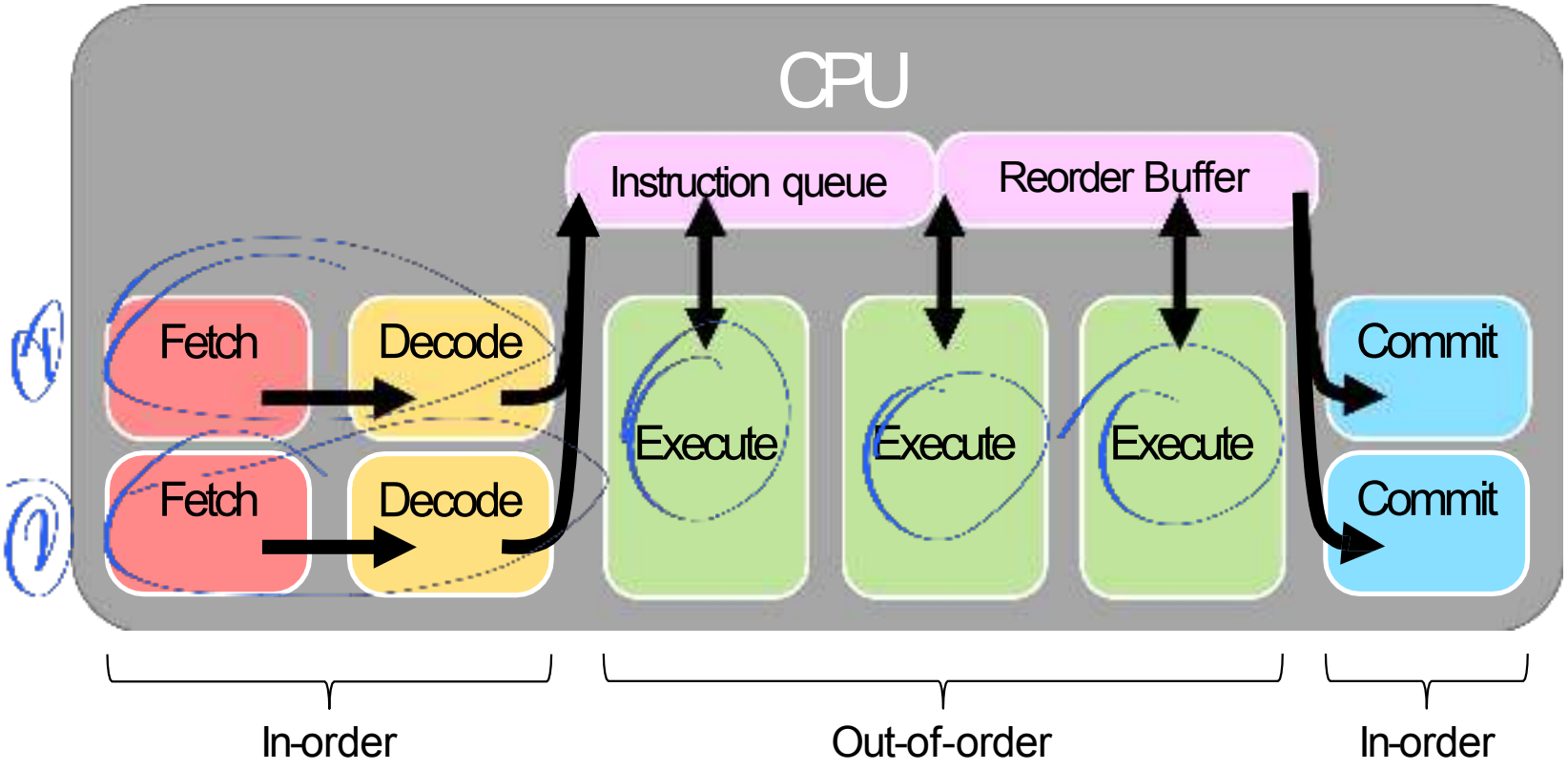
`ldr`, `mul` execute in 2 cycles
`cmp`, `bne` execute in 1 cycle
`mla` executes in 3 cycles



- **This isn't OoO… or even faster than a simple pipeline!**
- **Q: What went wrong?**
- **A: We're throughput-limited: can only exec 1 instrn**

High-level Superscalar OoO microarchitecture

- **Must increase *pipeline width* to increase ILP > 1 (2-way 3-issue)**

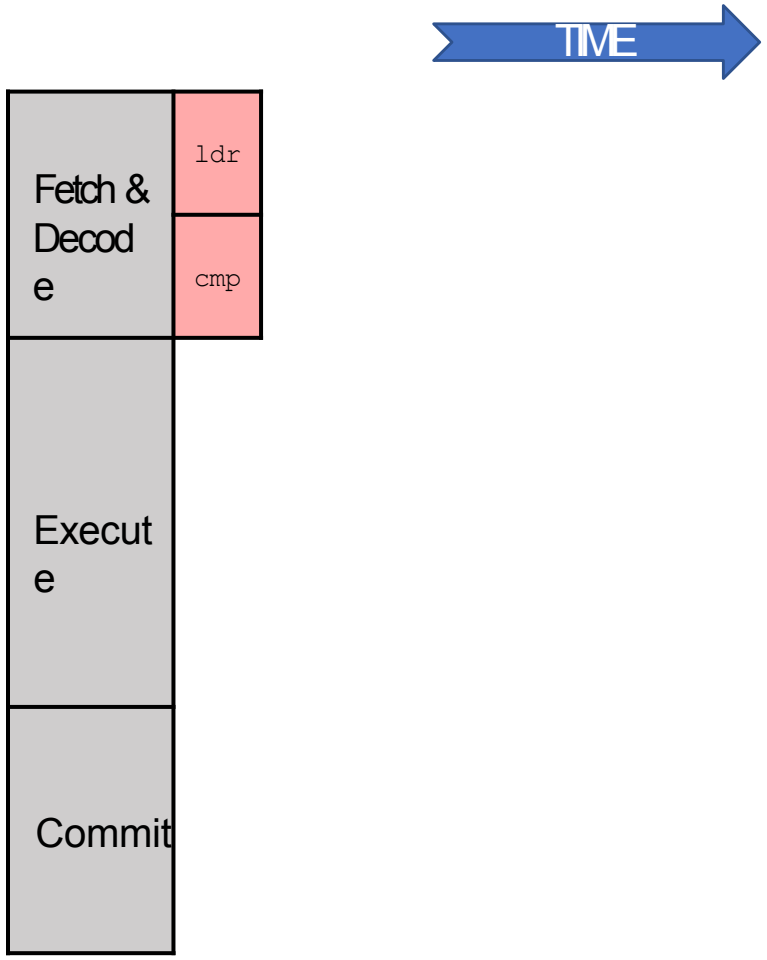


Focus on Execution, not Fetch & Commit

- Goal of OoO design is to only be limited by dataflow execution
- Fetch and commit are *over-provisioned* so that they (usually) do not limit performance
  → Programmers can (usually) ignore fetch/commit

- NOTEs: Programs with *inherently unpredictable* control flow will often be limited by fetch stalls (branch misprediction)
  - E.g., branching based on random data

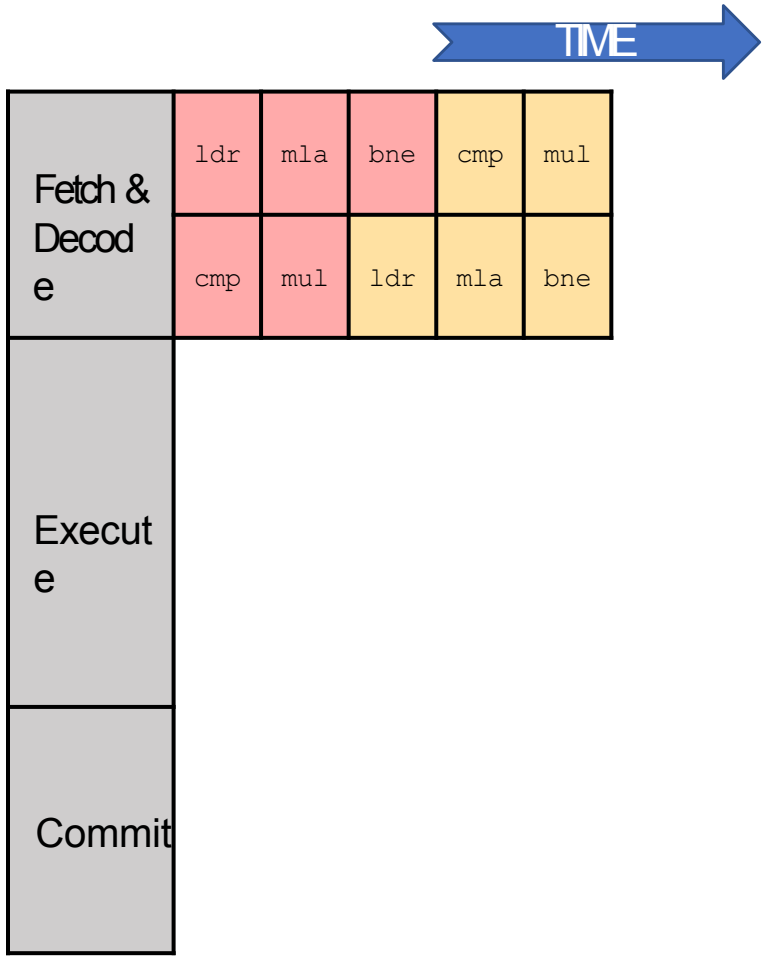
Example: Superscalar OoO polynomial evaluation




```
ldr     r5, [r3], #4
cmp     r1, r3
mla     r0, r4, r5, r0
mul     r4, r2, r4
bne     .L3
ldr     r5, [r3], #4
cmp     r1, r3
mla     r0, r4, r5, r0
mul     r4, r2, r4
bne     .L3
```

`ldr`, `mul` execute in 2 cycles
`cmp`, `bne` execute in 1 cycle
`mla` executes in 3 cycles

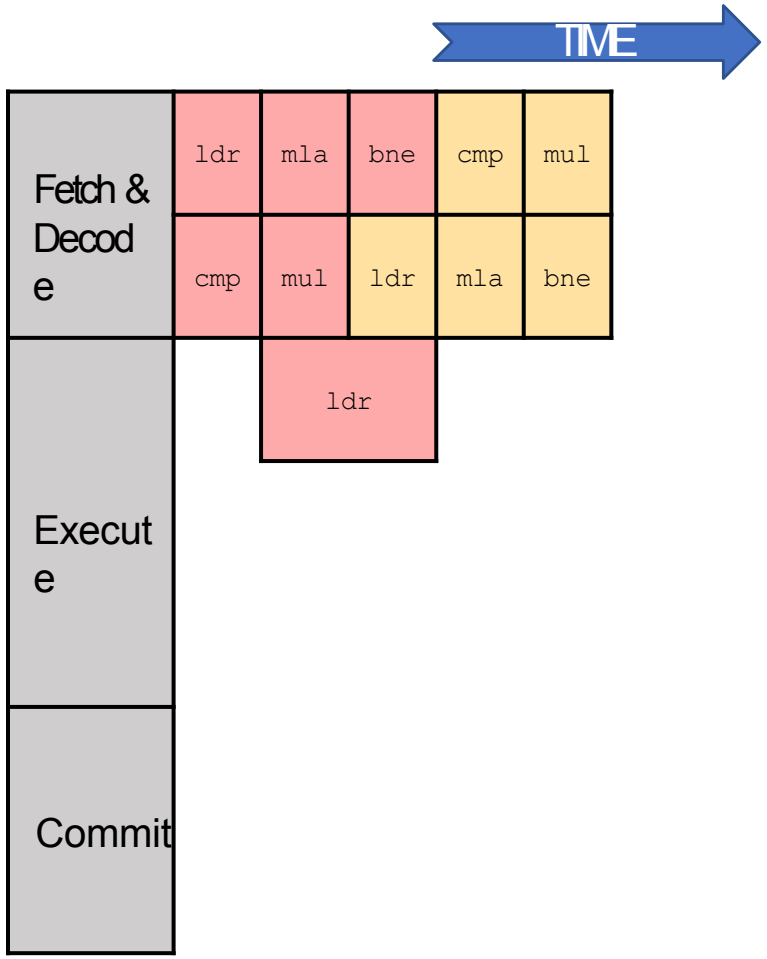
Example: Superscalar OoO polynomial evaluation




```
ldr     r5, [r3], #4
cmp     r1, r3
mla     r0, r4, r5, r0
mul     r4, r2, r4
bne     .L3
ldr     r5, [r3], #4
cmp     r1, r3
mla     r0, r4, r5, r0
mul     r4, r2, r4
bne     .L3
```

`ldr`, `mul` execute in 2 cycles
`cmp`, `bne` execute in 1 cycle
`mla` executes in 3 cycles

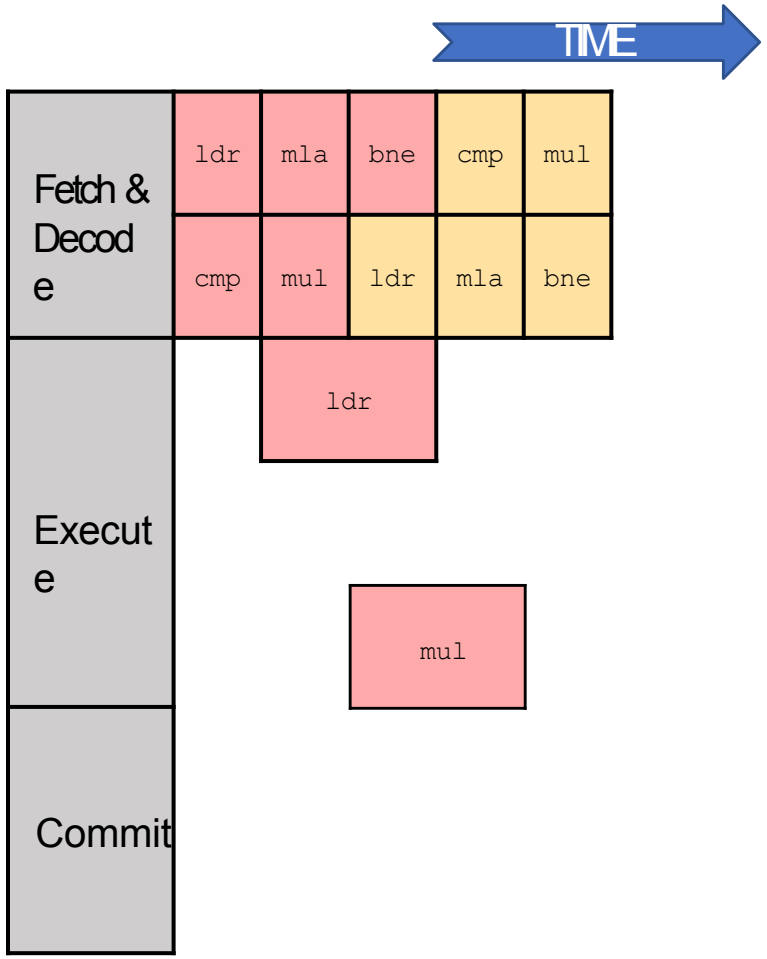
Example: Superscalar OoO polynomial evaluation




```
ldr     r5, [r3], #4
cmp     r1, r3
mla     r0, r4, r5, r0
mul     r4, r2, r4
bne     .L3
ldr     r5, [r3], #4
cmp     r1, r3
mla     r0, r4, r5, r0
mul     r4, r2, r4
bne     .L3
```

`ldr`, `mul` execute in 2 cycles
`cmp`, `bne` execute in 1 cycle
`mla` executes in 3 cycles

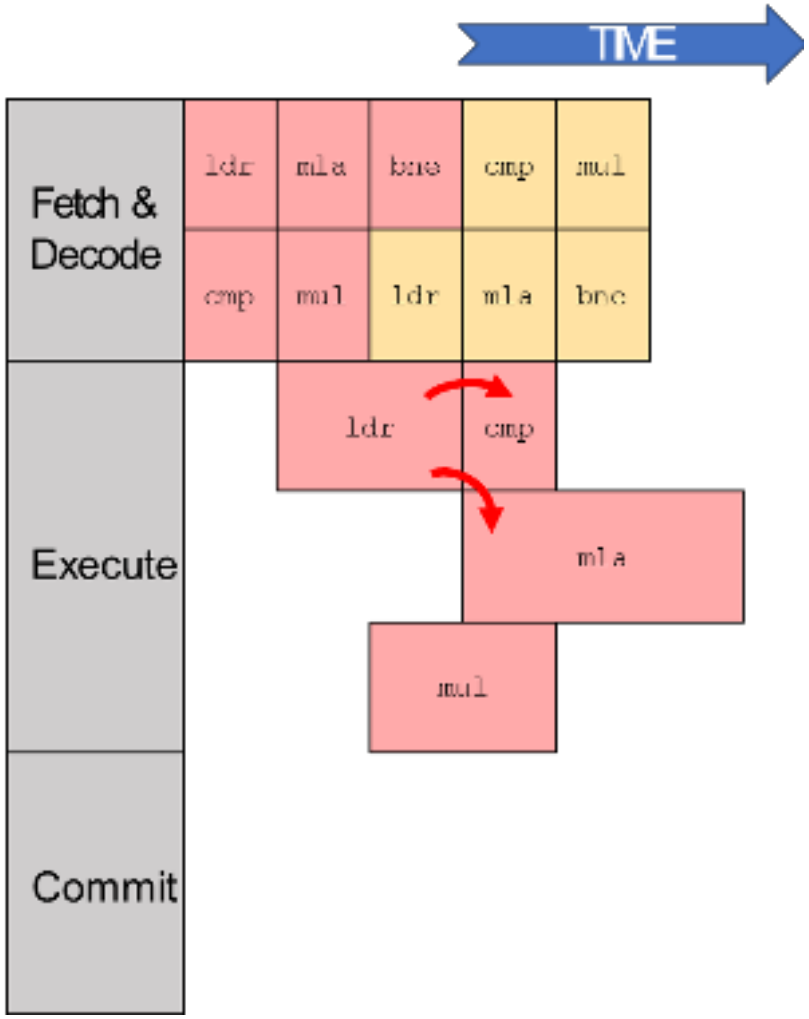
Example: Superscalar OoO polynomial evaluation




```
ldr     r5, [r3], #4
cmp     r1, r3
mla     r0, r4, r5, r0
mul     r4, r2, r4
bne     .L3
ldr     r5, [r3], #4
cmp     r1, r3
mla     r0, r4, r5, r0
mul     r4, r2, r4
bne     .L3
```

`ldr`, `mul` execute in 2 cycles
`cmp`, `bne` execute in 1 cycle
`mla` executes in 3 cycles

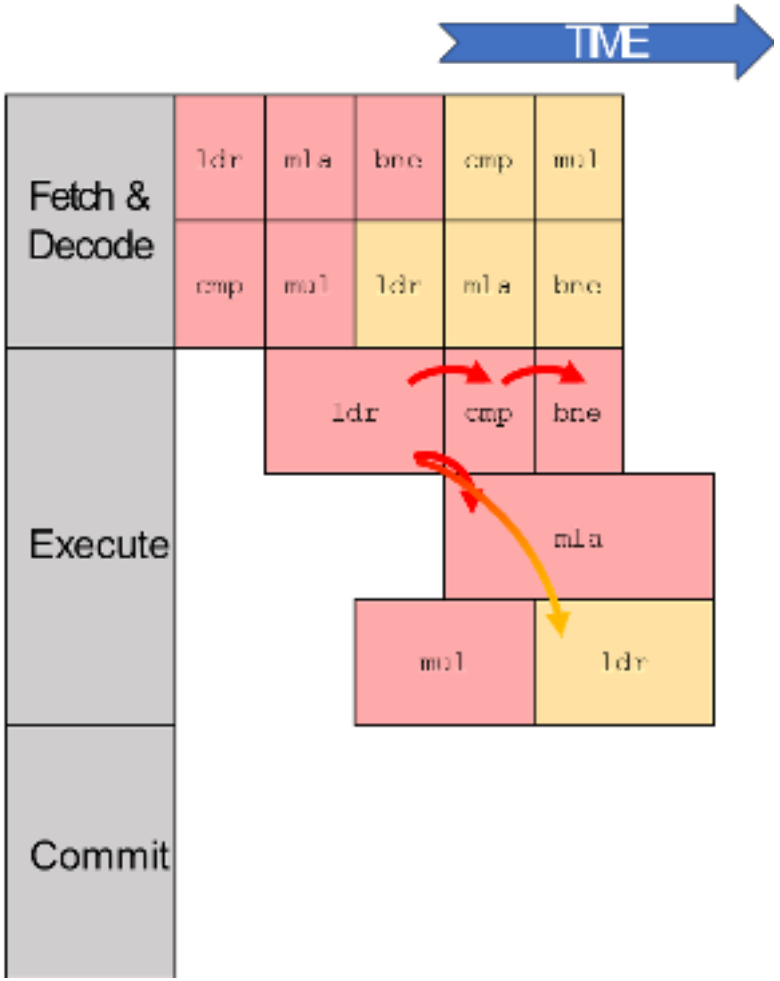
Example: Superscalar OoO polynomial evaluation




```
ldr     r5, [r3], #4
cmp     r1, r3
mla     r0, r4, r5, r0
mul     r4, r2, r4
bne     .L3
ldr     r5, [r3], #4
cmp     r1, r3
mla     r0, r4, r5, r0
mul     r4, r2, r4
bne     .L3
```

`ldr`, `mul` execute in 2 cycles
`cmp`, `bne` execute in 1 cycle
`mla` executes in 3 cycles

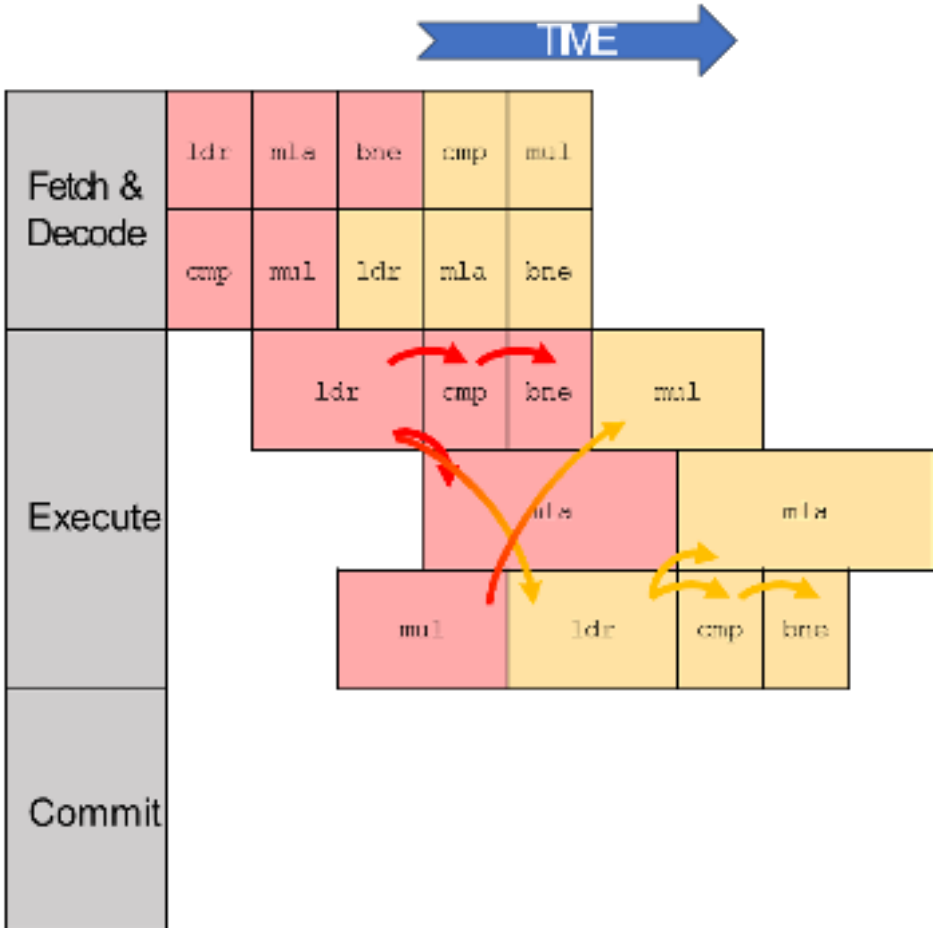
Example: Superscalar OoO polynomial evaluation




```
ldr     r5, [r3], #4
cmp     r1, r3
mla     r0, r4, r5, r0
mul     r4, r2, r4
bne     .L3
ldr     r5, [r3], #4
cmp     r1, r3
mla     r0, r4, r5, r0
mul     r4, r2, r4
bne     .L3
```

`ldr`, `mul` execute in 2 cycles
`cmp`, `bne` execute in 1 cycle
`mla` executes in 3 cycles

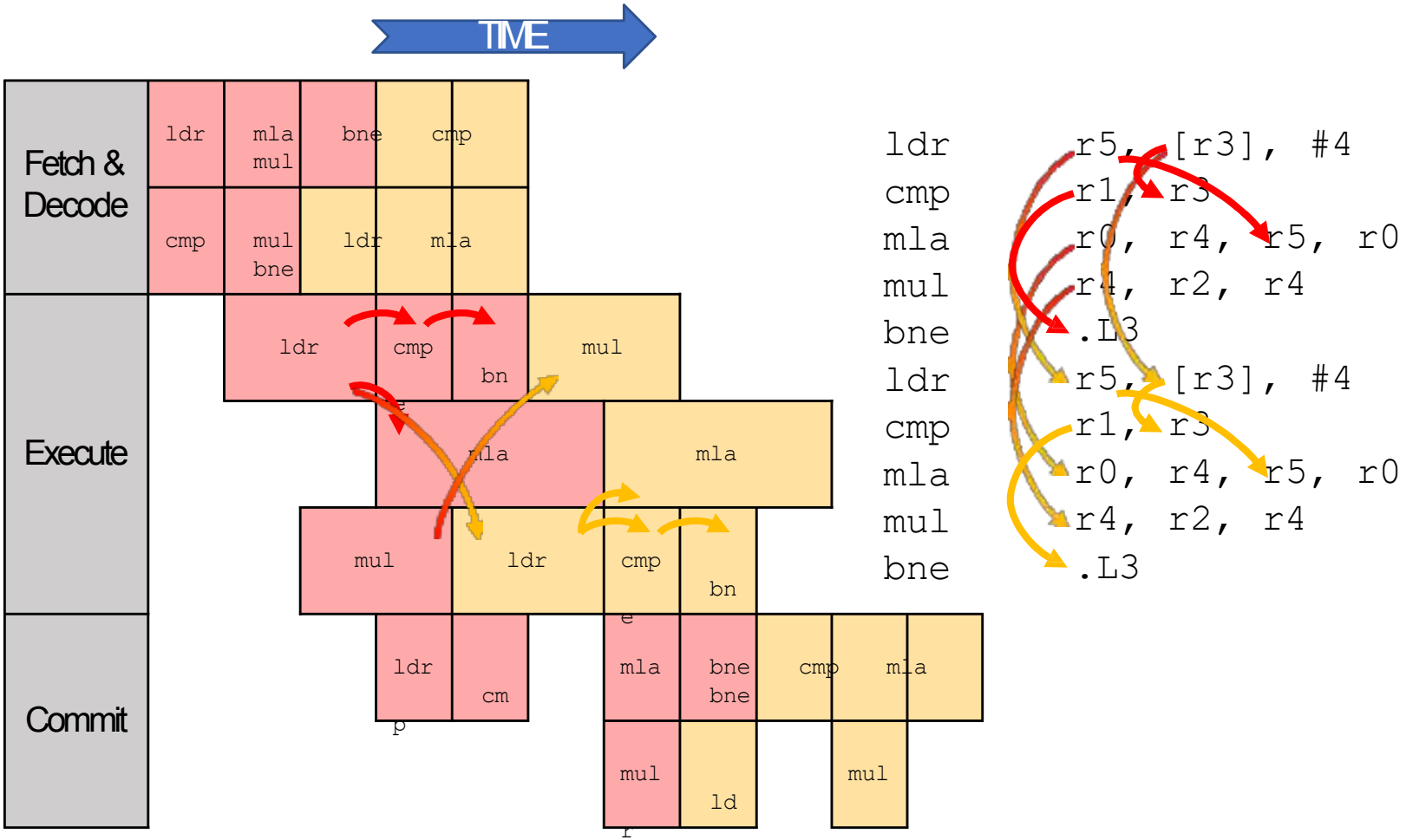
Example: Superscalar OoO polynomial evaluation



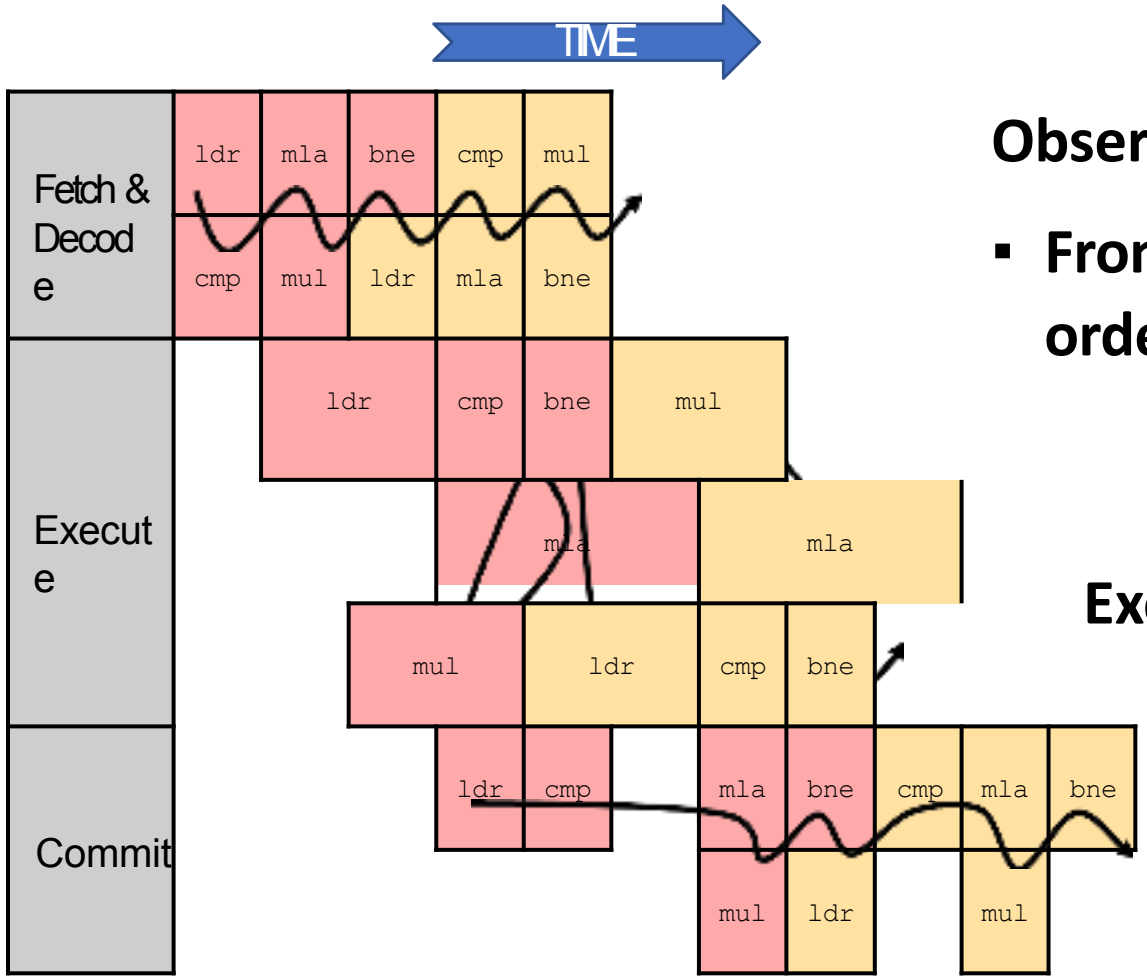

```
ldr     r5, [r3], #4
cmp     r1, r3
mla     r0, r4, r5, r0
mul     r4, r2, r4
bne     .L3
ldr     r5, [r3], #4
cmp     r1, r3
mla     r0, r4, r5, r0
mul     r4, r2, r4
bne     .L3
```

`ldr`, `mul` execute in 2 cycles
`cmp`, `bne` execute in 1 cycle
`mla` executes in 3 cycles

Example: Superscalar OoO polynomial evaluation



Example: Superscalar OoO polynomial evaluation

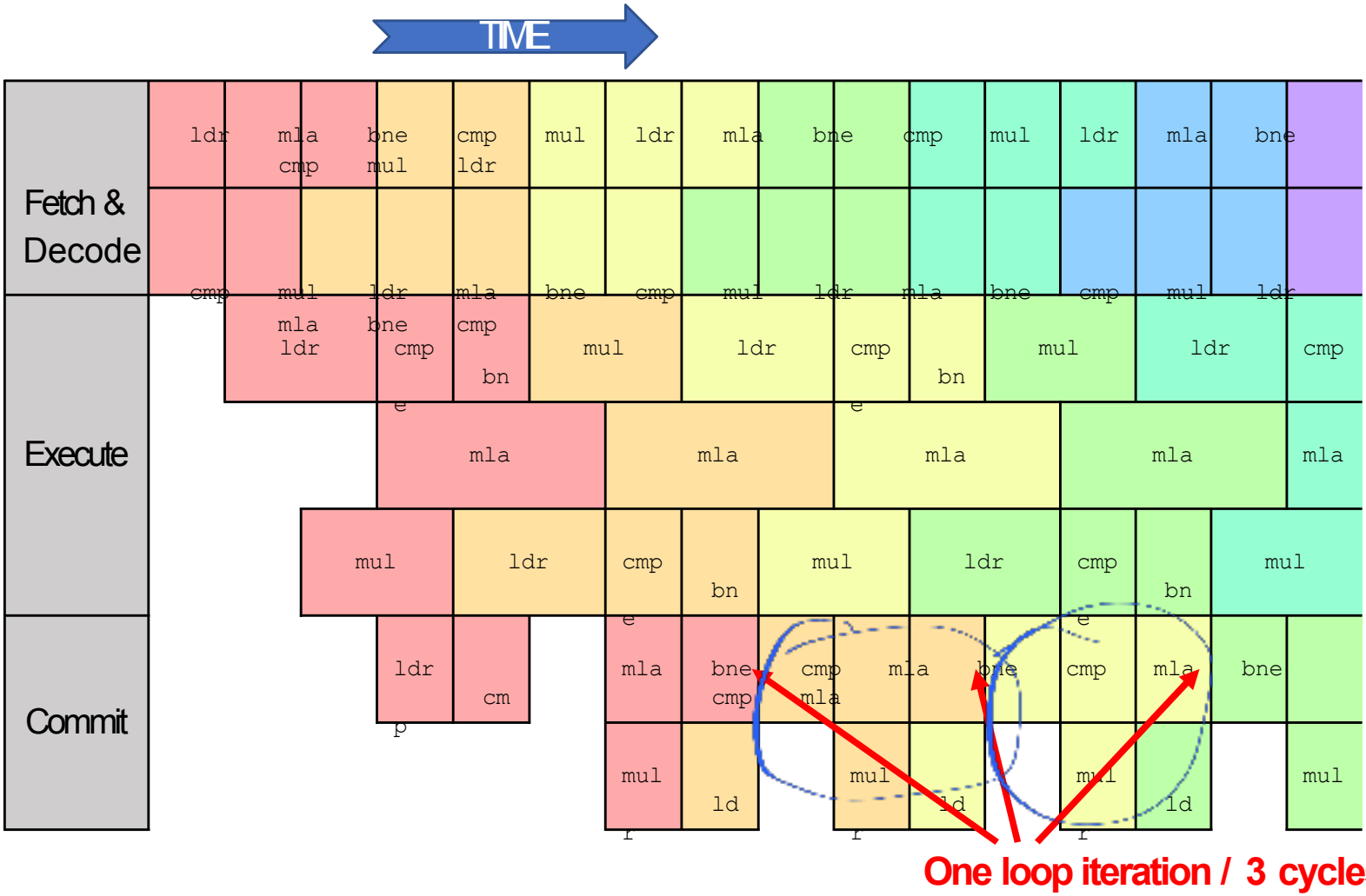


Observe:

- **Front-end & commit in-order (left-to-right)**

Execute out of order

Example: Superscalar OoO polynomial evaluation



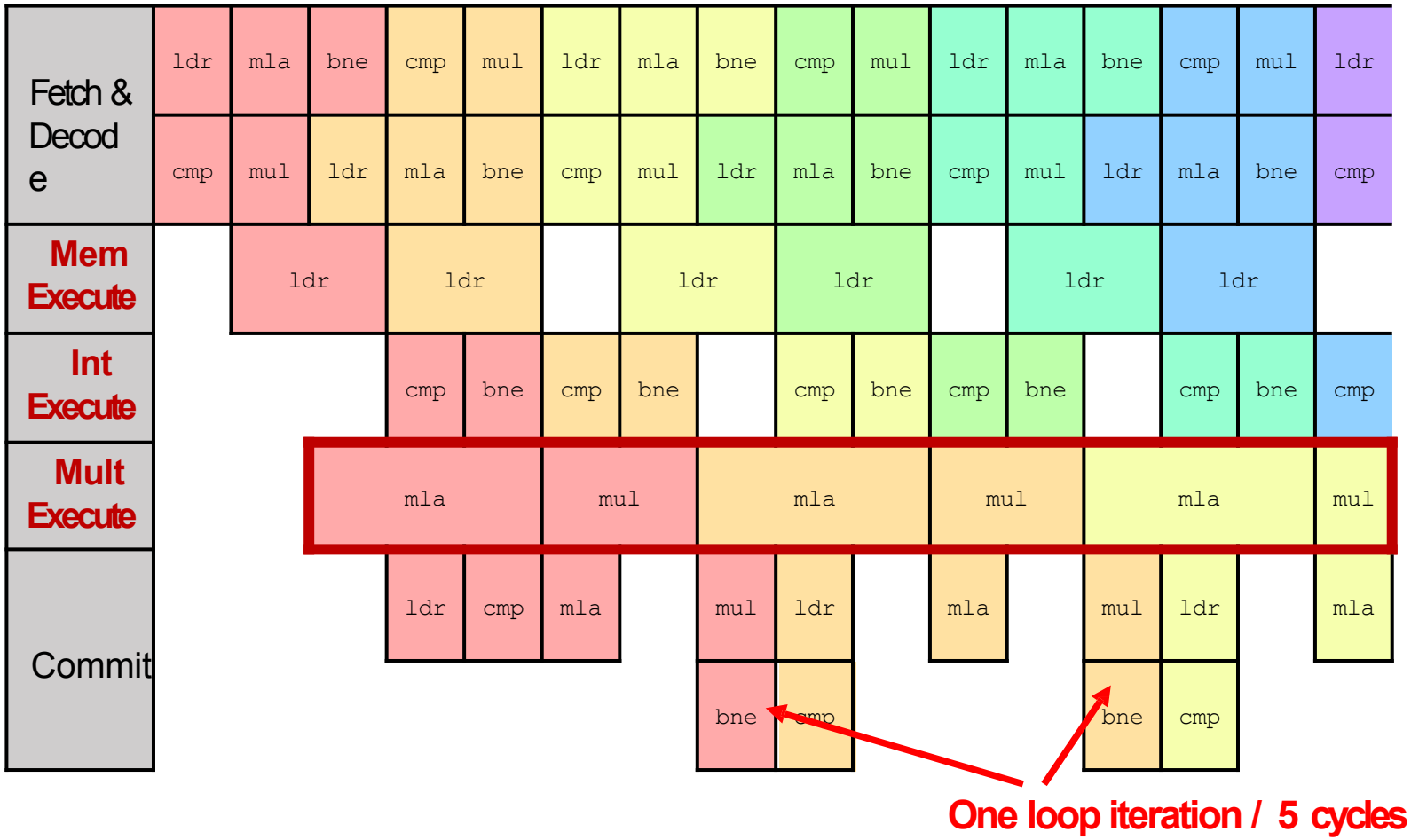
Structural hazards: Other throughput limitations

- However, execution units are specialized
  - Floating-point (add/multiply)
  - Integer (add/multiply/compare)
  - Memory (load/store)

- Processor designers must choose which execution units to include and how many

- *Structural hazard:* Data is ready, but instr'n cannot issue because no hardware is available

Example: Structural hazards can severely limit performance



Software Takeaway

- OoO is much less sensitive to "good code"
  - Better performance portability
  - Of course, compiler still matters

- OoO makes performance analysis much simpler
  - Throughput bound: Availability of *execution ports*
  - Latency bound: *"Critical path" latency*
  - Slowest gives good approximation of program perf

4.10.2 提高处理器线程级并行性的技术



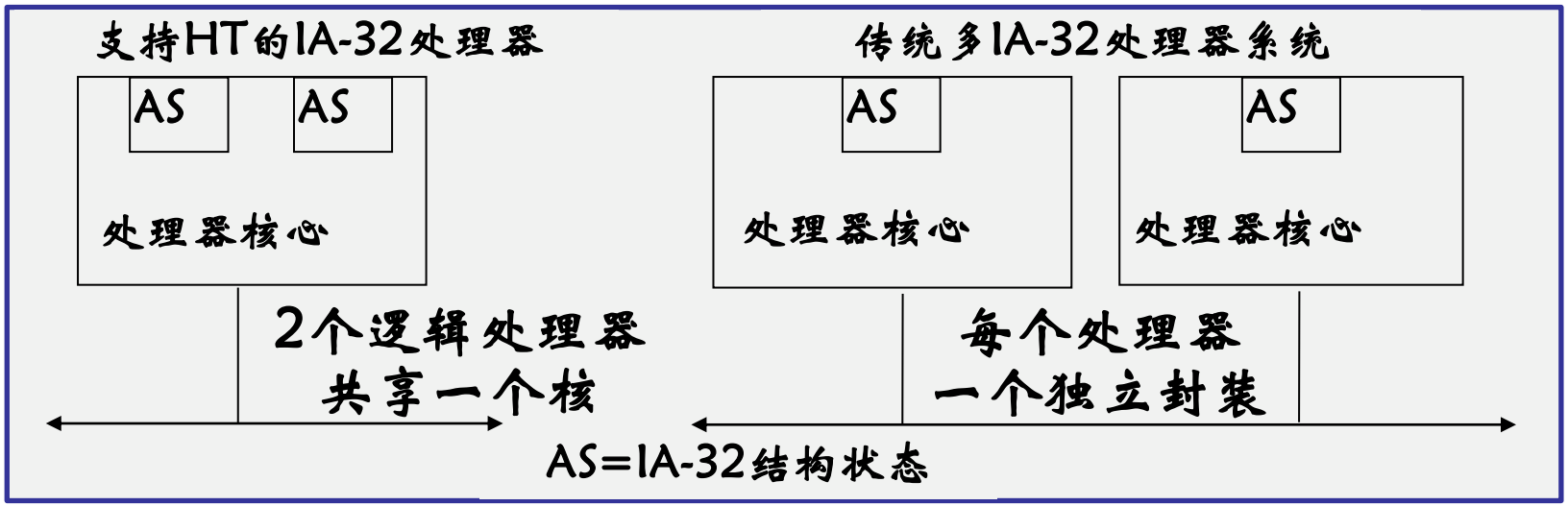
➢ 线程级并行性(TLP，Thread Level Parallel)

- 对于MIMD（多指令流多数据流），每个处理器执行自己的指令流
- 为了加大程序执行的并行力度，将程序划分为多个单一控制的执行流，每个执行流称之为一个线程。同一个进程中的不同线程共享数据空间，拥有自己的执行堆栈和程序计数器
- 提高系统整体的吞吐量
- 主要技术：超线程技术、多核技术

4.10.2 提高处理器线程级并行性的技术

1. 超线程(Hyper-Threading，HT)技术

- 2002年，Intel公司研发，首先应用于Xeon处理器
- 允许物理上单个处理器采用共享执行资源的方法同时执行两个或更多的分离代码流(线程)，又称软件多线程
- HT技术把单物理处理器模拟成2个或多个逻辑处理器
- 每个逻辑处理器都有独立的IA-32架构，即拥有自己的通用寄存器、段寄存器、控制寄存器、调试寄存器等
- 减少CPU的闲置时间，提高系统的资源利用率
- 逻辑处理器共享的资源包括执行引擎和系统总线接口



4.10.2 提高处理器线程级并行性的技术



1. 超线程(Hyper-Threading，HT)技术

- 2002年，Intel公司研发，首先应用于Xeon处理器
- 允许物理上单个处理器采用共享执行资源的方法同时执行两个或更多的分离代码流(线程)，又称软件多线程
- HT技术把单物理处理器模拟成2个或多个逻辑处理器

例如：

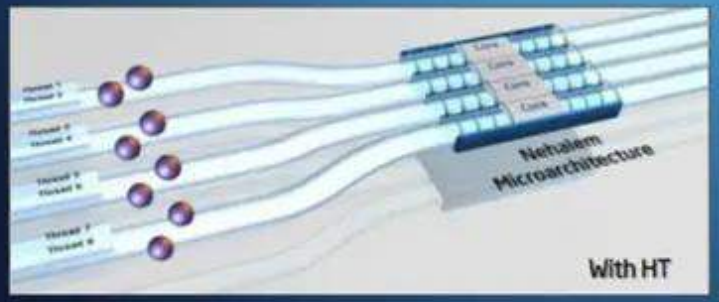
✓ 两个线程：一个整型运算、一个浮点运算

✓ 两个线程：一个需要运算、一个需要I/O



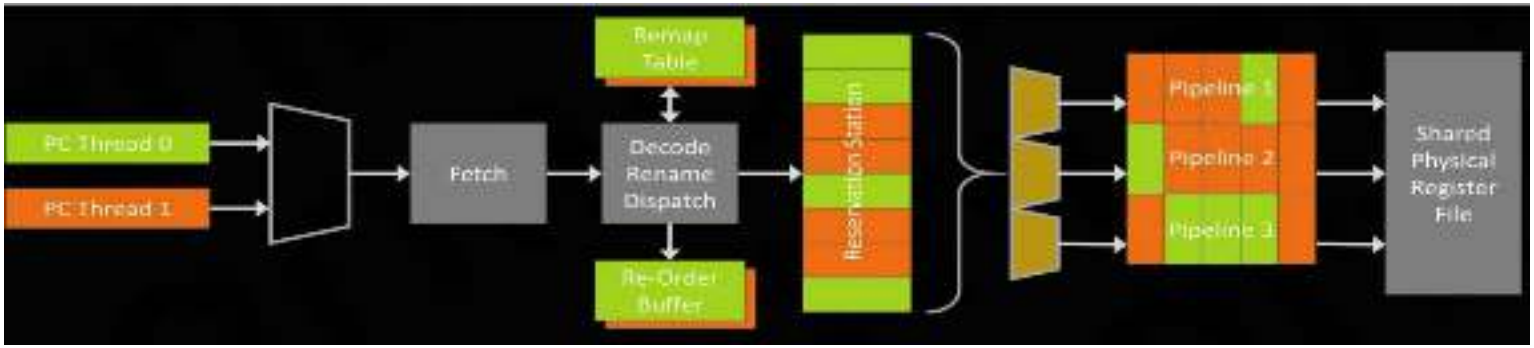
Intel® Hyper-Threading Technology For Better Multitasking[3]





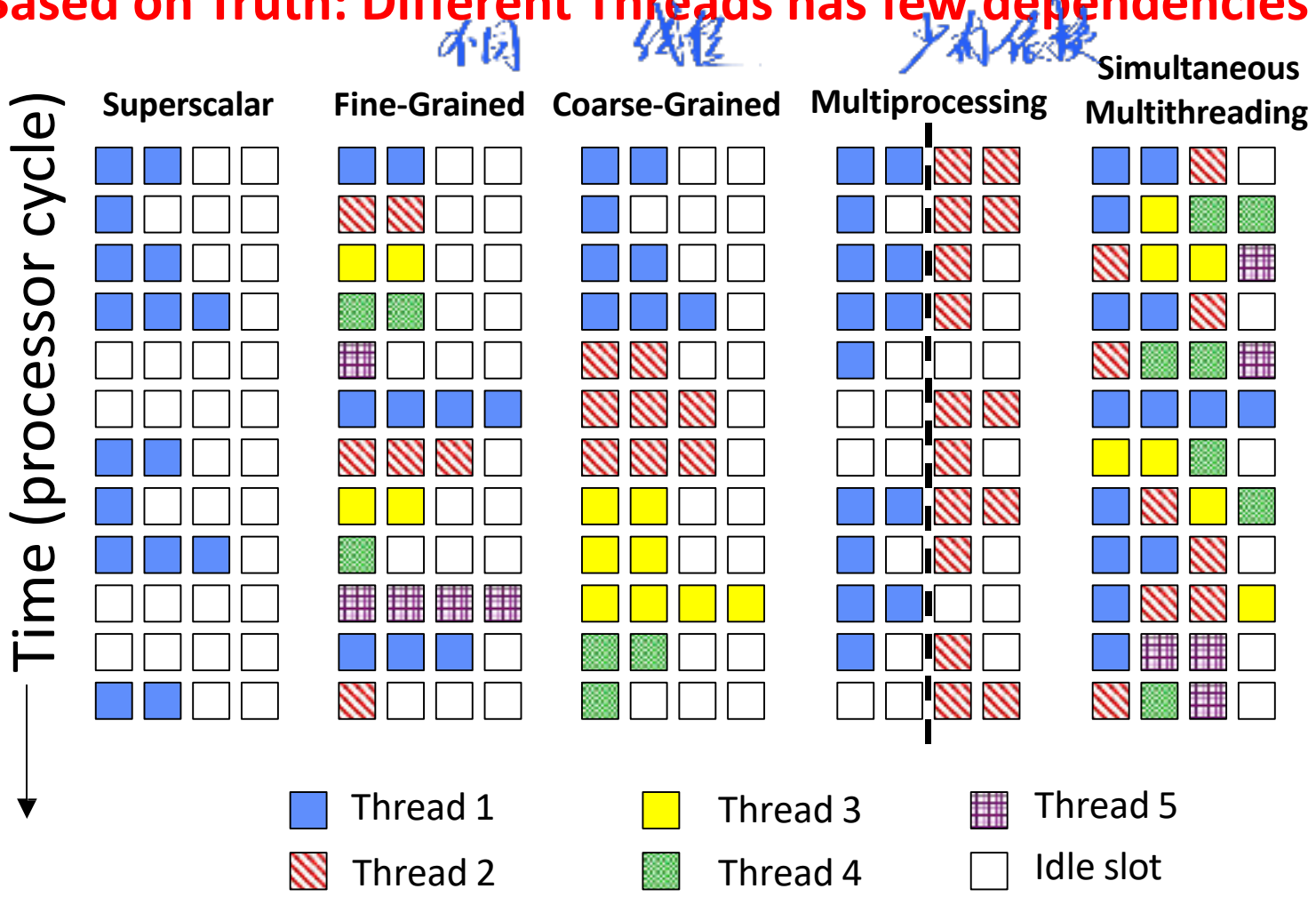
- Helps you do more and wait less
- Like having a freeway with multiple car lanes instead of just one
- With multiple lanes more cars can move more quickly

2 thread engines per core, enabling 8-way processing in 4-core systems. It means that a quad-core processor could run up to eight



HT为啥有效？

Based on Truth: Different Threads has few dependencies



4.10.2 提高处理器线程级并行性的技术



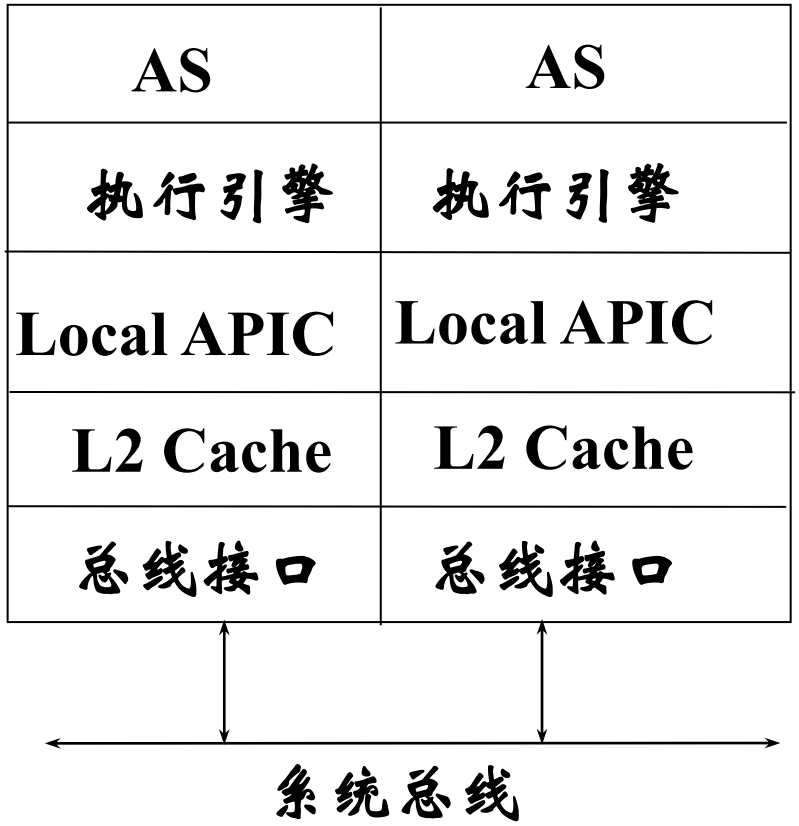
2. 多核(Multi-Core)技术——空间并行

- 2005年，Intel公司推出简单封装双核奔腾D；2006年发布酷睿(Core)架构处理器
- 通过在一个物理封装中集成多个分离的完整执行核来提供硬件多线程能力
- 每个完整的执行核拥有独立的指令集、执行单元，即不仅有自己的AS（Architecture State），还拥有自己的执行引擎，总线接口与L2 Cache等
- 本质
  - 硬件的冗余
  - 让不同的处理器并发执行不同的任务
- 效率与性能提升要比HT技术高得多

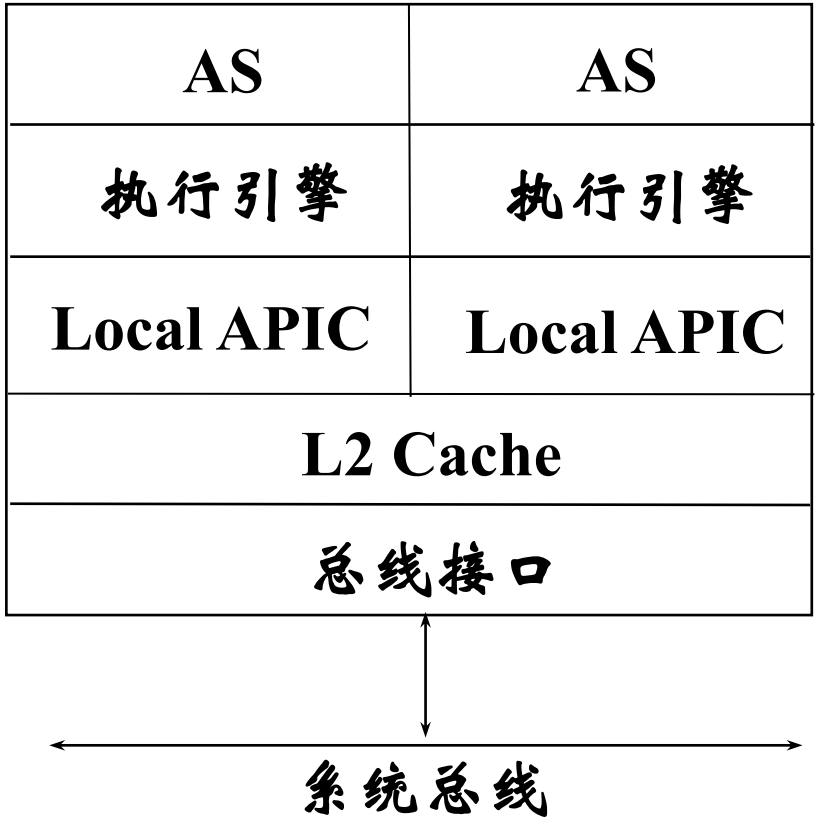
4.10.2 提高处理器线程级并行性的技术

➢ Intel的多核结构

Pentium D IA-32处理器

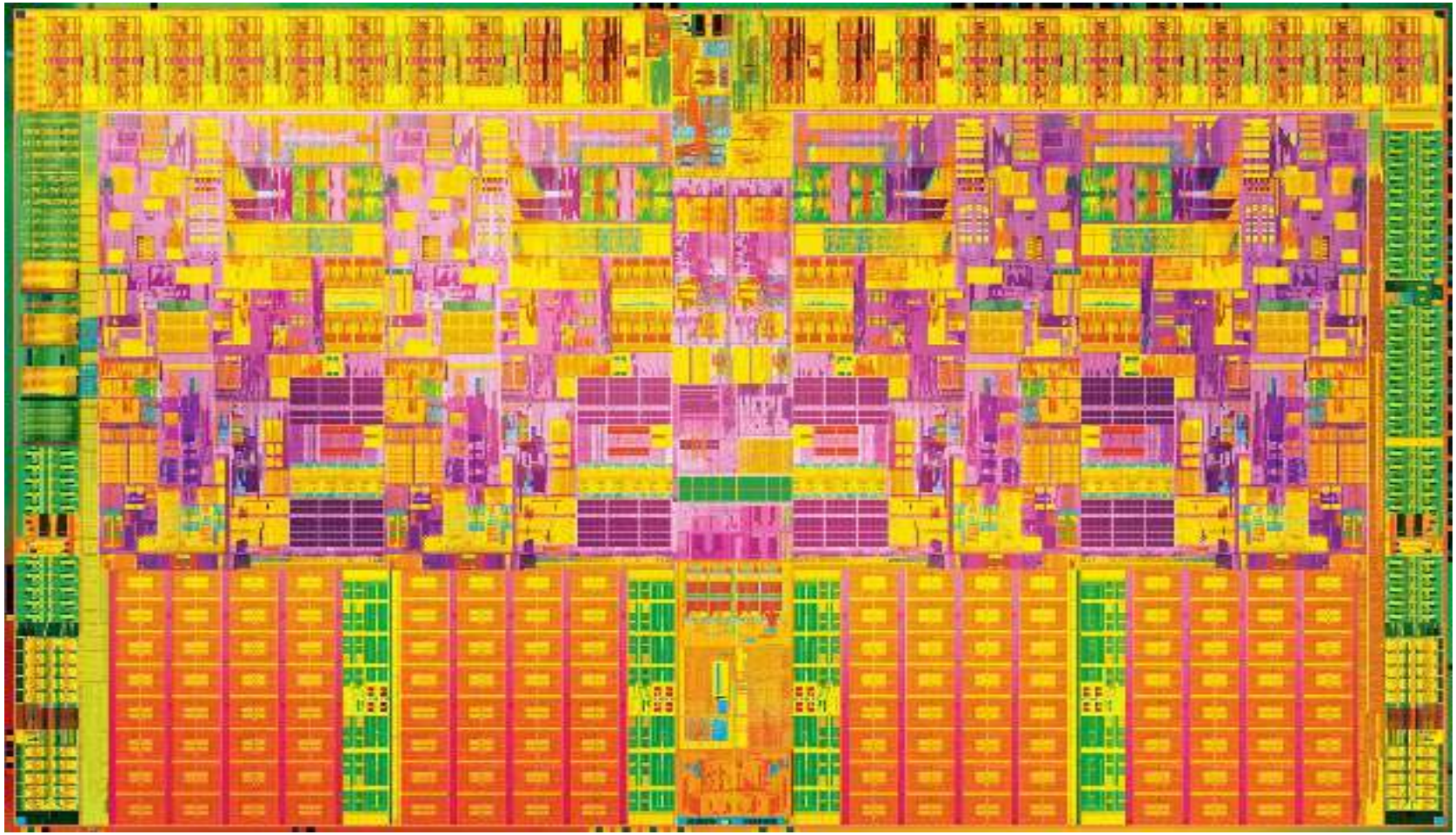


Core 2 Duo处理器

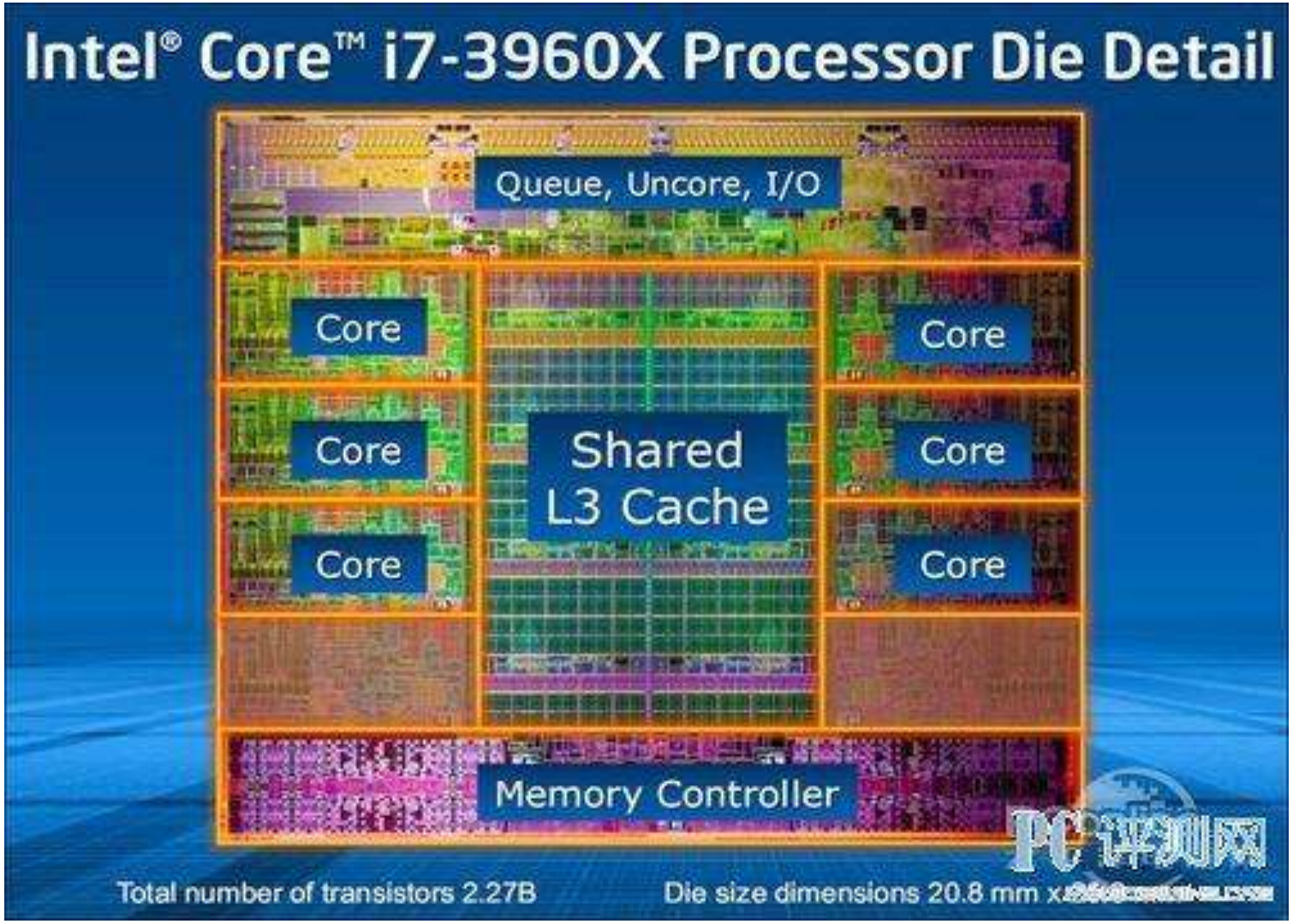


Intel的多核结构

Nehalem 微结构 Core i7 四核处理器

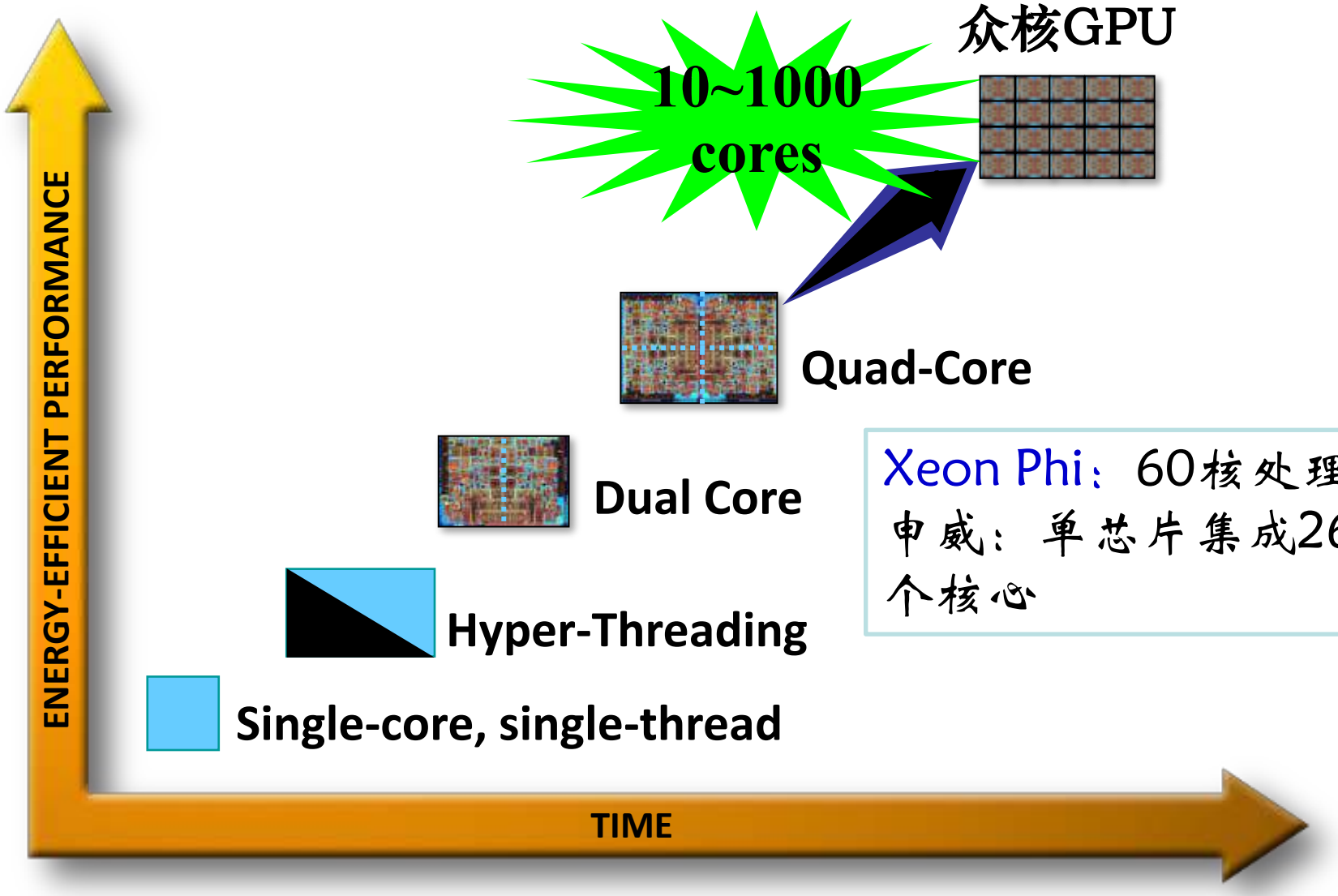


Intel的多核结构


Intel® Core™ i7-3960X Processor Die Detail
Queue, Uncore, I/O
Core
Core
Core
Core
Core
Core
Shared
L3 Cache
Memory Controller
Total number of transistors 2.27B
Die size dimensions 20.8 mm x
PC评测网

4.10.2 提高处理器线程级并行性的技术

□ 多核(Multi-Core)、众核(Many-Core)技术



Xeon Phi：60核处理器
申威：单芯片集成260个核心

补充：流水线数据相关性分析实例



例1：考察基本的MIPS 5级流水线中，执行如下指令序列时，请回答下列问题：

①指出是否存在数据冒险？属于何类型的数据相关？

②若没有转发机制，则如何增加NOP指令来消除数据冒险？

③若有全转发机制，是否全部解决数据冒险？若没有，则如何增加NOP指令来消除数据冒险？

	Instruction sequence
a.	SW R16, -100 (R6) LWR4, 8 (R16) ADDR5, R4, R4
b.	OR R1, R2, R3 OR R2, R1, R4 OR R1, R1, R2

流水线数据相关性分析实例



① 指出是否存在数据冒险？属于何类型的数据相关？

	Instruction sequence	dependence and their type
a.	I1:SW R16, -100 (R6) I2:LW R4, 8 (R16) I3:ADD R5, R4, R4	I2 与 I3 数据相关，对 R4 寄存器写后读（RAW）
b.	I1:OR R1, R2, R3 I2:OR R2, R1, R4 I3:OR R1, R1, R2	I1 与 I2 数据相关，对 R1 寄存器写后读（RAW） I1 与 I3 数据相关，对 R1 寄存器写后写（WAW） I2 与 I3 数据相关，对 R1 寄存器读后写（WAR） I1 与 I2 数据相关，对 R2 寄存器读后写（WAR） I2 与 I3 数据相关，对 R2 寄存器写后读（RAW）

流水线数据相关性分析实例

② 若流水线没有转发机制，则如何增加NOP指令来消除数据冒险？

	Instruction sequence with nop
a.	SW R16, -100 (R6)
	LW R4, 8 (R16)
	nop
	nop
	nop
	ADD R5, R4, R4

b.	OR R1, R2, R3
	nop
	nop
	OR R2, R1, R4
	nop
	nop
	OR R1, R1, R2

分析：在五级流水中，RAW会引起冒险，需要插入nop指令。如果约定寄存器堆操作是前半周期写，后半周期读，则只需插入2个nop指令；否则，需要插入3个nop指令。

谢 谢!



